Large Flow Language Models

Clearly Labeled Modified Copy

Publication package prepared October 30, 2026.

Status of this document

The pages after this cover reproduce a locally modified copy of the paper. It is not the original arXiv version or an official project release. The local author and affiliation block differs from the original; the research text, methods, experiments, bibliography, and figures are unchanged.

Original research authors

Arnav Chavan, Chanhyuk Lee, Udbhav Bamba, Jaehoon Yoo, Seunghoon Hong, Aditi Raghunathan, and Jinwoo Kim.

Credit for the original research remains with these authors. Cite the original arXiv author list, not the altered local byline.

Original source and license

Large Flow Language Models, arXiv:2610.28559v1, October 20, 2026.
`https://arxiv.org/abs/2610.28559v1`
Creative Commons Attribution 4.0 International (CC BY 4.0).
`https://creativecommons.org/licenses/by/4.0/`

Local changes

The October 28 local copy added Achintya Raya Polavarapu in place of Aditi Raghunathan and Jinwoo Kim, and removed Jaehoon Yoo and Seunghoon Hong from the displayed byline. It replaced Amazon with Meta and UC Berkeley with MIT, removed Carnegie Mellon University and the equal-advising marker, and listed MIT, Meta, and Georgia Institute of Technology for Achintya Raya Polavarapu. The changed MIT affiliation also appears beside Arnav Chavan; KAIST remains for Chanhyuk Lee.

These are document edits, not additional research contributions or evidence of approval by the original authors or listed institutions. This publication package adds the present cover and a modified-copy notice on each included page.

Document preparation

Achintya Raya Polavarapu. See the accompanying README and source archive for the change record and editable LaTeX files.

Large Flow Language Models

Arnav Chavan[*,1,2], Chanhyuk Lee[*,3], Udbhav Bamba[*,2]
Achintya Raya Polavarapu[1,2,4]

[1]Massachusetts Institute of Technology (MIT) [2]Meta [3]KAIST [4]Georgia Institute of Technology
[*]Equal contribution

Abstract. Continuous flow language models offer a path to parallel generation with tools for control and acceleration unavailable to discrete diffusion, yet effective techniques for building them at the billion-parameter scale remain less established. We present ***Lafa* (*La*rge *f*low l*a*nguage model)**, a fast, high-performing flow language model scaled to 8B parameters in a data-efficient manner. Lafa is a blockwise flow model with causal attention on the prefix and bidirectional, asynchronous flow within each block. This design enables efficient adaptation of pretrained autoregressive (AR) models and fast inference via prefix caching and blockwise parallel decoding. We train Lafa at 0.5B, 7B, and 8B scales with fewer than 30B tokens, more efficient than previous diffusion models, and evaluate it on instruction following, math, and coding, where the learned flow matches or surpasses the quality of AR and can be distilled into flow maps for one- and few-step generation. Notably, flow training preserves or even improves pretrained AR capabilities via a multi-token prediction effect, which we leverage in a self-speculative decoding scheme that verifies flow-based drafts autoregressively within a single model. This yields 2.54× throughput of an AR model at comparable quality, outperforming multi-token prediction and discrete diffusion methods. Our findings show that continuous flows can serve as a scalable foundation for language modeling.

Code: github.com/lafa-gh/large-flow-lms
Models: huggingface.co/lafa-hf/models

1 Introduction

Today's large language models are predominantly trained as autoregressive (AR) models that predict the next token from the preceding text [1–3]. The parallelism of the transformer architecture and teacher forcing [4, 5] allows these models to be trained efficiently at the scale of billions of parameters and trillions of data tokens. However, their generation is inherently sequential, producing one token per forward pass, which makes inference latency a bottleneck. To accelerate inference, prior work has explored approaches to parallel generation, namely multi-token prediction (MTP) [6–8], discrete diffusion models [9–11], and continuous flow language models [12–14].

Distinguished from the other approaches, flow language models perform parallel generation by integrating a continuous flow from noise to token representations [12]. By operating in a continuous space, they can leverage powerful techniques developed for controllable and accelerated generation of images and video, many of which are unavailable to discrete methods [13, 14]. Yet, existing flow language models have been demonstrated at small scales and on simple tasks [13–17], and

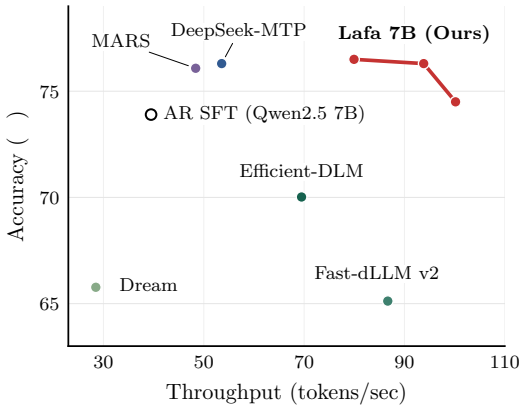


Figure 1: Large flow language models. Lafa 7B attains 2.02–2.54× the throughput of an AR model at a matched or better quality, outperforming existing MTP and diffusion-based approaches.

Modified document copy; original authors and local edits are listed on the cover.

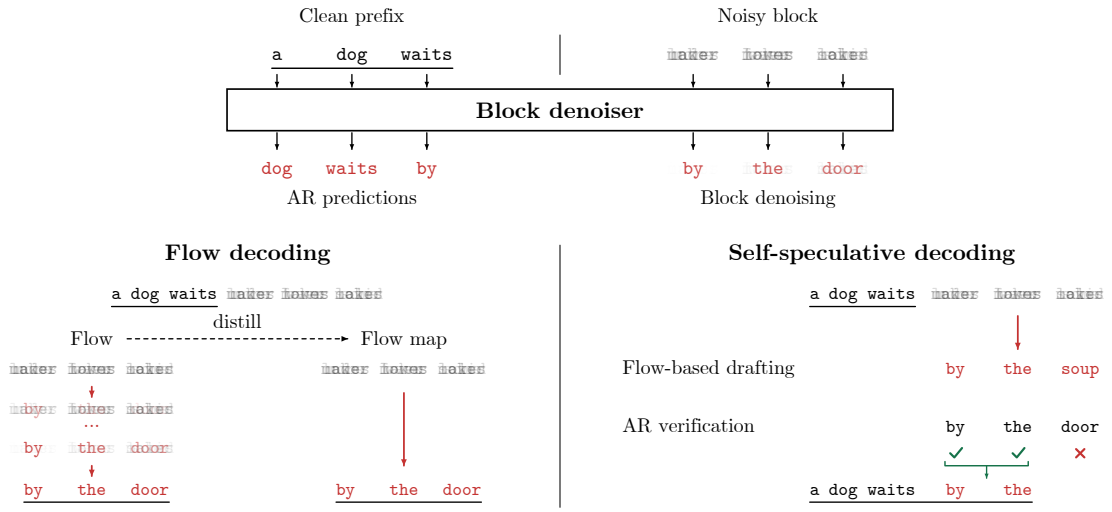


Figure 2: Overview. (Top) Lafa learns a block denoiser that also performs AR prediction on the prefix. (Bottom) Lafa supports flow decoding, which can be further distilled into a flow map for one-step generation. Together, AR and flow support self-speculative decoding, which accepts flow-based drafts from left to right via AR verification.

a prior attempt of scaling has identified challenges in training inefficiency [18]. Together, effective techniques for building performant flow language models at the billion-parameter scale remain less established.

In this paper, we present ***Lafa* (*La*rge *f*low l*a*nguage model)**, a fast and high-performing flow language model scaled to 8B parameters in a data-efficient manner. Motivated by the left-to-right directionality of text [19], we design Lafa as a blockwise flow model, with causal attention on the prefix, and bidirectional but left-to-right-biased asynchronous flow within each block. This brings its architecture and inductive bias close to AR models, enabling efficient training by adapting pretrained models. Further, this allows fast inference via prefix caching and parallel flow decoding, which can be further distilled into flow maps for one- and few-step generation. We train Lafa with blockwise denoising for the flow, along with AR prediction on the prefix, and find that flow training preserves or even enhances pretrained AR capabilities through a multi-token prediction effect. We leverage this in a self-speculative decoding scheme that verifies flow-based drafts autoregressively. With these techniques, we train Lafa across 0.5B, 7B, and 8B scales with fewer than 30B tokens, cheaper than comparable adaptation of discrete diffusion models that use up to 500B tokens [20, 21]. Across benchmarks spanning instruction following, reasoning, and coding, Lafa attains a quality–speed frontier that surpasses similarly sized AR, MTP, and discrete diffusion language models, as shown in Figure 1.

An overview of the approach is in Figure 2. Our main contributions are as follows:

- We introduce Lafa, a large flow language model that learns blockwise asynchronous flow, enabling efficient training by adapting pretrained AR models and fast inference through prefix caching and parallel flows.
- We identify key design choices for scalability, including the architecture and asynchronous flow formulation, training and flow map distillation objectives, and inference schemes including self-speculative decoding.
- We scale Lafa to 7–8B parameters with <30B tokens and show that its learned flow matches or surpasses the quality of AR and is distillable into flow maps, and its learning improves pretrained AR capabilities.
- By jointly leveraging the flow and AR capabilities for self-speculative decoding, Lafa attains 2.54× the throughput of a strong AR baseline at matched quality, outperforming existing parallel generation methods.

Our findings show that continuous flows offer a practical, scalable foundation for parallel language modeling.

Modified document copy; original authors and local edits are listed on the cover.

2 Large flow language models

In this section, we introduce Lafa, a class of scalable and high-performing large flow language models. We first formulate blockwise asynchronous flows, then describe the architecture, training, and inference procedures. See Appendix A for theoretical results and Appendix B for algorithmic details.

2.1 Block asynchronous flows

Blockwise formulation. We develop our method upon standard block-autoregressive factorization [22], as in block discrete diffusion [23], retaining causal generation and prefix caching across blocks. For vocabulary V, prefix $\mathbf{c} \in V^N$, and continuation $\mathbf{y} \in V^M$, we factorize into blocks of size B (assuming $B \mid M$):

$$p(\mathbf{y} \mid \mathbf{c}) = \prod_{r=0}^{M/B-1} p\big(\mathbf{y}^{rB+1:(r+1)B} \mid \mathbf{c}, \mathbf{y}^{\leq rB}\big). \tag{1}$$

Embedded flows. We embed each block $\mathbf{y} \in V^B$ using vocabulary embeddings $\mathbf{E} \in \mathbb{R}^{|V| \times d}$ as $\mathbf{x}_1 \coloneqq (\mathbf{E}_{\mathbf{y}^1}, \ldots, \mathbf{E}_{\mathbf{y}^B})^\top \in \mathbb{R}^{B \times d}$. This reduces the per-token dimension from $|V|$ of one-hot embeddings [13] to d, making training more scalable. Using flow matching [24, 25], we construct a conditional probability path $p_t(\cdot \mid \mathbf{c})$, $t \in [0,1]$, from Gaussian noise $\mathbf{x}_0 \sim \mathsf{N}(0, \sigma^2 I)$ with $\sigma > 0$ to the distribution of embedded data $\mathbf{x}_1$.

Asynchronous flows. Standard flow matching uses the interpolant $I_t \coloneqq (1-t)\mathbf{x}_0 + t\mathbf{x}_1$, where $I_t \sim p_t$, with a shared time t for all tokens. To introduce a left-to-right bias within each block to match the inherent directionality of text [19], we use tokenwise time [26–30], allowing earlier positions to progress faster. A tokenwise time is a function $\tau : [0,1] \to [0,1]^B$, differentiable and increasing in each coordinate, mapping a base time t to a tokenwise time $\tau^i(t)$ for each $i = 1, ..., B$, satisfying $\tau(0) = \mathbf{0}$ and $\tau(1) = \mathbf{1}$. Given τ, we can construct a probability path $p_t(\cdot \mid \mathbf{c})$ in base time t through an interpolant I_t defined positionwise [27]:

$$I_t^i \coloneqq (1 - \tau^i(t))\, \mathbf{x}_0^i + \tau^i(t)\, \mathbf{x}_1^i, \qquad i \in [B], \qquad I_t \sim p_t(\cdot \mid \mathbf{c}), \qquad t \in [0,1]. \tag{2}$$

For each given $\mathbf{c}$, the probability path $p_t(\cdot \mid \mathbf{c})$ admits a velocity field $b_t(\cdot; \mathbf{c}) : \mathbb{R}^{B \times d} \to \mathbb{R}^{B \times d}$ that transports a block of noise to clean data embeddings through the following ODE [27], which can be used for generation:

$$\dot{\mathbf{x}}_t = b_t(\mathbf{x}_t; \mathbf{c}), \qquad \mathbf{x}_0 \sim p_0, \qquad \mathbf{x}_t \sim p_t(\cdot \mid \mathbf{c}), \qquad t \in [0,1]. \tag{3}$$

If τ progresses faster for the earlier tokens (i.e., $\tau^i(t) > \tau^j(t)$ for $i < j$), both interpolation and generation will reveal earlier tokens faster. In practice, we learn τ from data instead of fixing it, with details in Section 2.2.

Block denoiser. Instead of learning the asynchronous velocity $b_t(\cdot; \mathbf{c})$ directly, following related work [12, 13], we learn a block denoiser $D_t(\cdot; \mathbf{c}) : \mathbb{R}^{B \times d} \to (\Delta^{|V|-1})^B$ that predicts clean tokens via tokenwise classification:

$$D_t(\mathbf{x}; \mathbf{c})^i \coloneqq p(\mathbf{y}^i = \cdot \mid \mathbf{c}, I_t = \mathbf{x}), \qquad i \in [B], \tag{4}$$

where $\Delta^{|V|-1}$ is the space of probability vectors of length $|V|$. Notably, learning the denoiser recovers the velocity through the following identity that involves embedding interpolation, which we prove in Appendix A.1:

$$b_t(\mathbf{x}; \mathbf{c})^i = \frac{\dot{\tau}^i(t)}{1 - \tau^i(t)} (D_t(\mathbf{x}; \mathbf{c})^i \mathbf{E} - \mathbf{x}^i). \tag{5}$$

The formula scales the direction from noisy input $\mathbf{x}$ to its denoising $D_t(\mathbf{x}; \mathbf{c})\mathbf{E}$ by the tokenwise rate of progress $\dot{\tau}(t)$, thereby implementing the asynchronous progress prescribed by tokenwise time τ.

Modified document copy; original authors and local edits are listed on the cover.

2.2 Learning the flow from an autoregressive model

As our formulation keeps a causal structure, we learn the blockwise flow by adapting a pretrained AR model f into a block denoiser. The challenge is parallel denoising of continuous, noisy embeddings while retaining AR prediction and prefix caching. Our adaptation combines tokenwise time conditioning, block-bidirectional attention, and constrained updates to pretrained embeddings with joint AR and denoising training.

Tokenwise time conditioning. We feed f the embedded prefix $\mathbf{c}$ followed by the noisy block $\mathbf{x}$, and condition each position on its corruption level through adaptive layer normalization (adaLN) [31]. Prefix positions always receive time 1, indicating the absence of noise, while each block position $i \in [B]$ receives its tokenwise time $\tau^i(t)$. This lets a single forward pass handle both the clean prefix and asynchronous denoising within the block. We initialize adaLN to the identity, so that adding time conditioning initially leaves the pretrained computation unchanged. We denote the resulting model by $f_t(\mathbf{x}; \mathbf{c})$.

Block-bidirectional attention. We make attention bidirectional within the noisy block, allowing partially denoised later tokens to inform earlier ones (Figure 3). Block positions also attend to the entire clean prefix, whose attention remains causal and excludes the noisy block [32, 33]. Together with conditioning prefix positions on time 1, this makes their keys and values independent of the block state $(\mathbf{x}, t)$, allowing reuse across denoising steps and subsequent blocks. The prefix outputs retain their AR role: $f_t(\mathbf{x}; \mathbf{c})^i$ predicts $\mathbf{c}^{i+1}$ for $i < N$, and the first block token $\mathbf{y}^1$ for $i = N$. We write these block-independent predictions as $f(\mathbf{c})^i$ and use them to verify the denoiser's drafts in self-speculative decoding (Section 2.3). This requires retaining or improving pretrained AR capabilities, which we achieve through training. We find that bidirectional attention within blocks outperforms causal (Table 3).

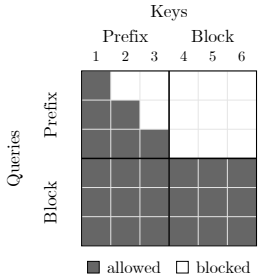


Figure 3: Attention mask for block-bidirectional attention.

Shared and learned embeddings. We initialize the vocabulary embeddings $\mathbf{E}$ from the pretrained model, share them between the prefix and the embedded flow, and train them jointly with the denoiser. Unconstrained updates, however, can inflate embedding norms relative to the fixed noise scale σ, making denoising easier by reducing the effective corruption [12, 13]. We therefore fix the L2 norm of each embedding $\mathbf{E}_v$ to its pretrained value and learn only its direction, allowing adaptation to the flow while preserving the embedding scale. This outperforms both frozen and unconstrained embeddings (Table 3).

Training loss. We train the block denoiser $\hat{D}_t(\mathbf{x}; \mathbf{c})^i := f_t(\mathbf{x}; \mathbf{c})^{N+i}$ jointly with AR on the prefix. For each training pair $(\mathbf{c}, \mathbf{y})$, we draw $t \sim \mathsf{U}[0, 1]$ and noise $\mathbf{x}_0 \sim p_0$, and build I_t via (2) with $\mathbf{x}_1^i = \mathbf{E}_{\mathbf{y}^i}$. We minimize

$$\mathcal{L}(\hat{D}) := \mathbb{E}_{\mathbf{c},\mathbf{y},t,\mathbf{x}_0}\left[\underbrace{\sum_{i=1}^{N} -\log f(\mathbf{c})^i_{\mathbf{c}^{i+1}}}_{\text{AR cross entropy}} + \underbrace{\sum_{i=1}^{B} -\log \hat{D}_t(I_t; \mathbf{c})^i_{\mathbf{y}^i}}_{\text{denoising cross entropy}} + \underbrace{\sum_{i=1}^{B} \left|\hat{D}_t(I_t; \mathbf{c})^i \mathbf{E} - \mathbf{E}_{\mathbf{y}^i}\right|^2}_{\text{embedding squared error}}\right], \tag{6}$$

where $\mathbf{c}^{N+1} := \mathbf{y}^1$, and the AR and denoising predictions share the transformer parameters. The denoising cross entropy supervises token probabilities (4), while the squared error supervises the mean embedding that determines the flow velocity (5). Combining these denoising losses improves flow accuracy, and we unexpectedly find that AR supervision improves it further (Table 3). Notably, joint training also improves AR accuracy over matched AR SFT (Section 3.2), consistent with the benefits of multi-token prediction [6, 8, 34].

Parallel training. To avoid recomputing overlapping prefixes, we train all blocks of a sequence in parallel. Following prior work on block diffusion [23, 32, 33, 35], we pair each clean training sequence with a copy partitioned into noisy blocks of size B (Figure 4). Clean positions attend causally, while each noisy block attends only to the preceding clean tokens and bidirectionally within itself. A single forward pass then produces AR predictions for the clean sequence and denoising predictions for every block (details in Appendix B.1).

Modified document copy; original authors and local edits are listed on the cover.

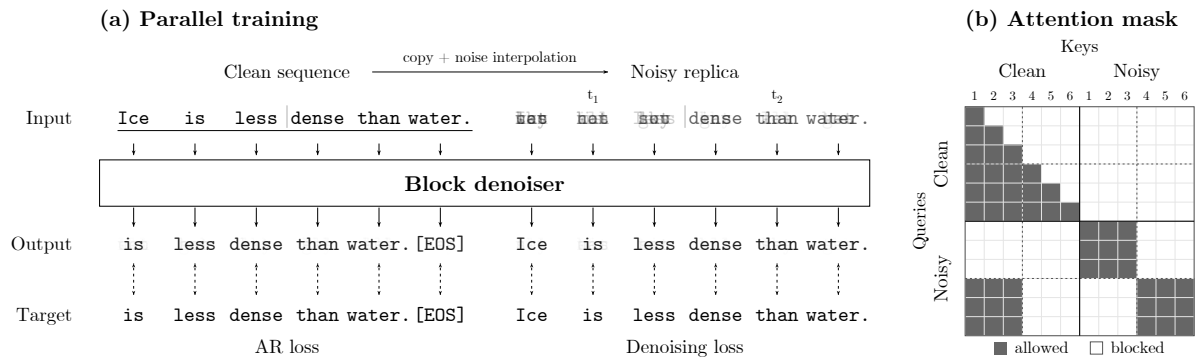


Figure 4: Parallelized training over a sequence composed of two blocks of size three. The uncorrupted sequence uses causal attention and the AR loss, while each noisy block attends to the preceding clean tokens and bidirectionally within itself, and uses the denoising losses.

Learning tokenwise time. We learn the tokenwise time $\tau : [0,1] \to [0,1]^B$ from the denoising loss during training. Inspired by the loss-aware time warping of CDCD [12], we construct each τ^i as the inverse CDF of the normalized denoising loss across noise levels, estimated online for position i (details in Appendix B.1). This slows progress where the loss is high, so uniform sampling and discretization of base time t allocate more training samples and integration steps to difficult noise levels. The learned schedules generally satisfy $\tau^i(t) > \tau^j(t)$ for $i < j$ (Figure 5), with earlier positions denoised ahead of later ones, even though this left-to-right bias is not manually imposed.

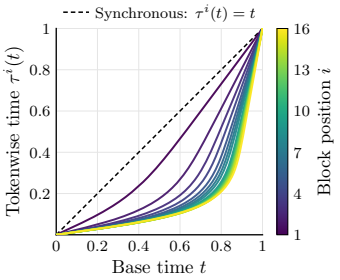


Figure 5: Tokenwise time for each block position learned by Lafa 7B.

Self-conditioning. Following prior work [14, 15, 36], we use self-conditioning [37] to improve sample quality. With a fixed probability during training, we obtain a denoising prediction without gradients, feed it back as an additional input, and compute the loss on a second forward pass (details in Appendix B.2). During flow decoding, each Euler step reuses the previous step's prediction for conditioning, requiring no added forward.

Flow map distillation. Flow decoding requires multiple denoiser evaluations per block. To reduce this cost, we distill the learned flow into a flow map $X_{s,t}$ that transports a state directly from time s to t [38]. For this, we make a novel extension of simplex-valued two-time denoisers [13] to asynchronous flows over vocabulary embeddings. Specifically, $\delta_{s,t}(\mathbf{x};\mathbf{c}) \in (\Delta^{|V|-1})^B$ predicts token probabilities, which we map through $\mathbf{E}$ before interpolating with the noisy state $\mathbf{x}$:

$$X_{s,t}(\mathbf{x};\mathbf{c})^i := \frac{1-\tau^i(t)}{1-\tau^i(s)}\mathbf{x}^i + \frac{\tau^i(t)-\tau^i(s)}{1-\tau^i(s)}\,\delta_{s,t}(\mathbf{x};\mathbf{c})^i\mathbf{E}, \qquad i \in [B], \quad 0 \le s < t \le 1. \tag{7}$$

This retains categorical predictions for distillation while transporting states in embedding space. We initialize the student $\hat{\delta}_{s,t}$ from the frozen teacher $\hat{D}_s$, adding an identity-initialized adaLN for the destination time t. Following semigroup-based distillation [13, 36], we match the student's diagonal predictions at $s = t$ to the teacher $\hat{D}_s$ and its off-diagonal predictions at $s \neq t$ to a stop-gradient target formed by composing two shorter student transitions. We use KL losses for both terms and additionally distill the teacher's prefix AR predictions. The student preserves prefix caching and supports generation in as few as one step through $\hat{X}_{0,1}$. The derivation and full objective are in Appendix A.2.

Modified document copy; original authors and local edits are listed on the cover.

2.3 Inference schemes

We generate each block by integrating the learned flow or evaluating its distilled flow map. We further accelerate generation through self-speculative decoding, using the retained AR predictor f to verify flow-based drafts. All schemes reuse the prefix key-value (KV) cache and extend it with newly committed tokens.

Flow and flow map decoding. Starting from $\hat{\mathbf{x}}_0 \sim p_0$, we take K steps on the grid of base times $t_k = k/K$, which induces an asynchronous temporal grid $\tau(t_k) \in [0,1]^B$. With $\mathbf{p}_k := \hat{D}_{t_k}(\hat{\mathbf{x}}_{t_k}; \mathbf{c})$, an Euler step gives

$$\hat{\mathbf{x}}^i_{t_{k+1}} = \hat{\mathbf{x}}^i_{t_k} + \frac{\tau^i(t_{k+1}) - \tau^i(t_k)}{1 - \tau^i(t_k)}(\mathbf{p}^i_k \mathbf{E} - \hat{\mathbf{x}}^i_{t_k}), \qquad \hat{\mathbf{y}}^i = \mathsf{argmax}(\mathbf{p}^i_{K-1}), \qquad i \in [B], \tag{8}$$

where the final prediction determines the generated tokens $\hat{\mathbf{y}}$. Each step reuses the previous prediction for self-conditioning. We further strengthen prefix conditioning with prefix guidance [39], analogous to classifier-free guidance [40] (details in Appendix B.2). This adds one prefix-free evaluation per step, which is efficient as it processes only the B block tokens. The distilled flow map model replaces each Euler step with a learned jump, which amounts to using $\hat{\delta}_{t_k,t_{k+1}}$ in place of $\hat{D}_{t_k}$ in (8):

$$\hat{\mathbf{x}}_{t_{k+1}} = \hat{X}_{t_k,t_{k+1}}(\hat{\mathbf{x}}_{t_k}; \mathbf{c}), \qquad \hat{\mathbf{y}}^i = \mathsf{argmax}\big(\hat{\delta}_{t_{K-1},1}(\hat{\mathbf{x}}_{t_{K-1}}; \mathbf{c})^i\big), \qquad i \in [B]. \tag{9}$$

Setting $K = 1$ gives one-step block generation through $\hat{X}_{0,1}$.

Self-speculative decoding. We draft a block with one Euler step in tokenwise time (8), without prefix guidance. Since $\tau^i(0) = 0$ and $\tau^i(1) = 1$, this reduces to a single denoiser evaluation from $\hat{\mathbf{x}}_0 \sim p_0$:

$$\mathbf{p}^i := \hat{D}_0(\hat{\mathbf{x}}_0; \mathbf{c})^i, \qquad \hat{\mathbf{x}}^i_1 = \mathbf{p}^i \mathbf{E}, \qquad \hat{\mathbf{y}}^i = \mathsf{argmax}(\mathbf{p}^i), \qquad i \in [B]. \tag{10}$$

One-step drafts can be inconsistent across positions, so we verify them with the retained AR predictor f [41]. We commit the longest draft prefix agreeing with greedy AR predictions, and if the first token disagrees, we instead commit $\mathsf{argmax}(f(\mathbf{c})^N)$. The resulting output therefore matches greedy AR decoding with f. To improve efficiency, following fused drafting and verification schemes [41], we verify the previous draft while generating candidate next drafts with one-step flow decoding in a single forward pass. Here, each candidate conditions on a possible accepted prefix, and after verification, we keep the candidate corresponding to the verified acceptance length (details in Appendix B.3). As an alternative to self-speculation, a draft may be accepted by confidence thresholding [33], which we detail in Appendix C.3.

3 Experiments

In our experiments, we evaluate whether Lafa learns flows that reach AR quality through data-efficient adaptation, and whether flow map distillation improves few-step generation (Section 3.1). We then examine how flow training affects AR accuracy (Section 3.2) and use the retained AR predictor for self-speculative decoding, measuring both parallelism and wall-clock throughput (Section 3.3). Finally, we ablate the main training and inference choices (Section 3.4). Evaluation details are in Appendix C.

Models and data. We build Lafa at 0.5B and 7B by adapting the corresponding Qwen2.5 Instruct checkpoints [42] as described in Section 2. At 8B, we use Qwen3 8B Base [43], as further training of its heavily post-trained Instruct counterpart degraded performance in our experiments. We train all models on Dolci-Instruct-SFT [44] for 5–15 epochs (9–27B tokens), using at least 18× fewer training tokens than the reported budgets of Dream, LLaDA, and Efficient-DLM at comparable scales [20, 21, 45]. We use 0.5B for ablations, 7B for the main results, and 8B for scale-matched comparisons with additional baselines.

Modified document copy; original authors and local edits are listed on the cover.

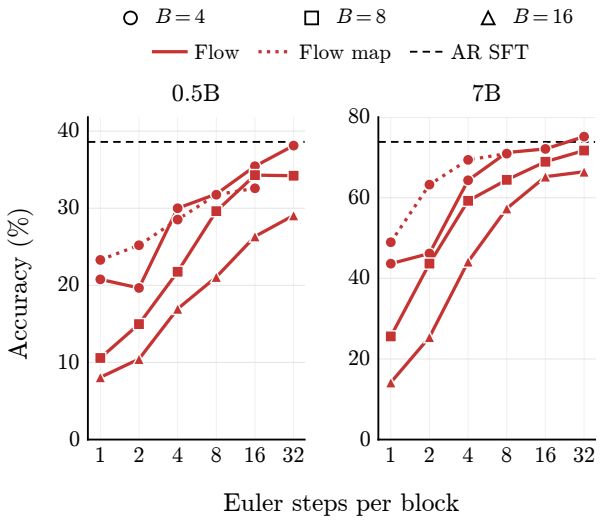


Figure 6: Flow and flow map decoding results. Accuracy averaged over IFEval, GSM8K, and HumanEval.

Table 1: Single-token decoding accuracy (%) ↑ averaged across all six benchmarks. † denotes matched initialization, data, and training.

Method	0.5B	7B	8B
AR SFT†	28.7	58.4	56.8
MTP† [6]	25.9	53.9	–
DeepSeek-MTP† [8]	30.2	<u>60.0</u>	–
MARS† [33]	<u>30.4</u>	59.7	–
SBD† [46]	27.6	–	–
BD3-LM† [23]	22.8	–	–
Fast-dLLM v2† [48]	–	49.3	–
Fast-dLLM v2 [48]	–	54.4	–
Dream [20]	–	55.6	–
LLaDA [45]	–	–	43.5
Efficient-DLM [21]	–	–	**63.5**
Lafa (Ours)†	**31.6**	**60.9**	<u>63.1</u>

Baselines. We compare against multi-token prediction, block-parallel decoding, and discrete diffusion methods that represent existing approaches for parallel language modeling (Section 4). At 0.5B, we evaluate MTP [6], DeepSeek-MTP [8], MARS [33], SBD [46], BD3-LM [23], and PTP [47]. At 7–8B, we additionally compare against Fast-dLLM v2 [48], Dream [20], LLaDA [45], and Efficient-DLM [21]. Baselines marked † are trained by us with the same AR initialization, data, and training budget as Lafa, providing controlled comparisons. The remaining baselines use released checkpoints with different training data and budgets. We also fine-tune each AR initialization on the same data to obtain AR SFT, our reference for both quality and speed. Unless stated otherwise, we use each method's native decoding protocol.

Benchmarks. We evaluate on six benchmarks spanning instruction following, reasoning, and code generation: IFEval (0-shot) [49], BBH (3-shot) [50], MMLU-Pro (0-shot) [51], GPQA (0-shot) [52], GSM8K (0-shot) [53], and HumanEval (0-shot) [54]. These downstream tasks assess generation quality without relying on generative perplexity, whose limitations are documented by Franca and Tong [55]. All methods are evaluated with lm-evaluation-harness [56] using each benchmark's default prompts. We report prompt-level strict accuracy for IFEval, pass@1 for HumanEval, and accuracy for the remaining tasks. Generation is capped at 1024 tokens per sample, except 512 for GSM8K and 640 for HumanEval. We report six-benchmark macro-averages for single-token accuracy and self-speculative accuracy and TPF (Tables 1 and 2 and Figure 7). For flow and flow map decoding, throughput, and ablations, we average over IFEval, GSM8K, and HumanEval, covering instruction following, math, and coding, considering their more costly evaluations.

3.1 Flow and flow map decoding

We first evaluate flow generation with block sizes $B \in \{4, 8, 16\}$, varying the number of Euler integration steps in (8). We also test whether flow map distillation improves few-step generation. Figure 6 summarizes the results (per-benchmark scores in Appendix C.1). Across scales and block sizes, flow accuracy generally improves with more integration steps, demonstrating effective iterative refinement. With sufficient steps, flow decoding with block size four approaches AR SFT accuracy at 0.5B and surpasses it at 7B. At a fixed number of steps, larger blocks generally trade accuracy for greater parallelism. For flow map decoding, we distill the models with block size four and observe improved accuracy in the one- and two-step regimes. The gains are largest at 7B, where the two-step student approaches the four-step teacher; the student also outperforms the teacher at four steps and reaches similar accuracy at eight steps. Nevertheless, a quality gap remains relative to flow decoding with more integration steps. We leave better incorporation of self-conditioning and prefix guidance into flow map distillation to future work.

Modified document copy; original authors and local edits are listed on the cover.

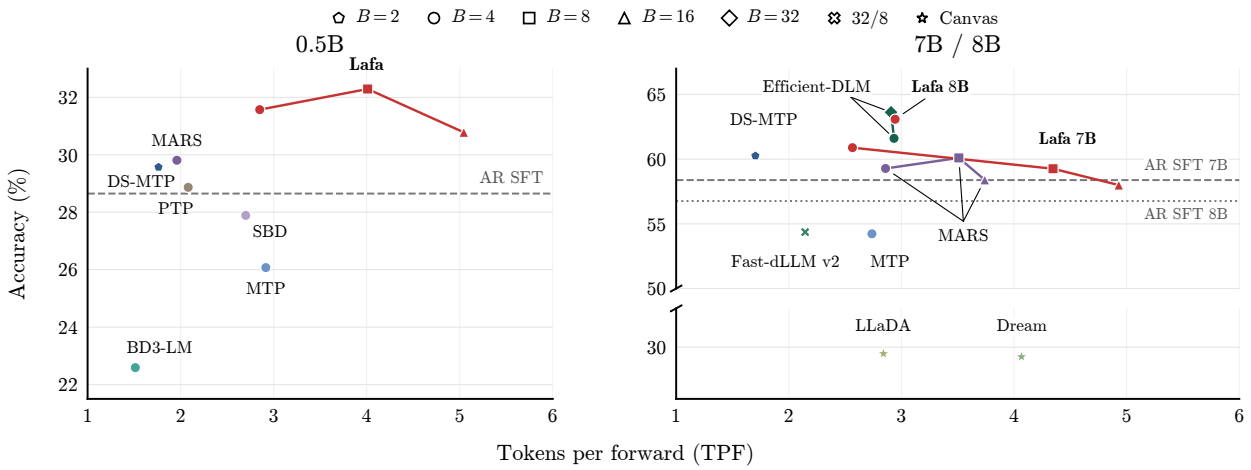


Figure 7: Self-speculative decoding. Accuracy ↑ vs. TPF ↑ averaged over all six benchmarks. In the right panel, Efficient-DLM and LLaDA are 8B and the other baselines are 7B. "Canvas" denotes iterative completion over the full sequence, while "32/8" denotes decoding 32-token blocks in four 8-token sub-blocks. Further details in Appendix C.1.

Table 2: Self-speculative decoding accuracy (%) ↑ and TPF ↑.

		IFEval		BBH		MMLU-Pro		GPQA		GSM8K		HumanEval		Average	
Method	B	Acc.	TPF	Acc.	TPF	Acc.	TPF	Acc.	TPF	Acc.	TPF	Acc.	TPF	Acc.	TPF
0.5B scale															
Qwen2.5 SFT	–	48.4	1.0	26.3	1.0	11.9	1.0	17.9	1.0	32.0	1.0	35.4	1.0	28.7	1.0
Lafa (Ours)	4	55.1	2.2	27.6	3.3	**12.1**	3.1	21.9	2.9	36.1	2.7	36.6	2.9	31.6	2.9
	8	**55.3**	2.7	**27.9**	5.1	10.0	5.5	**23.7**	4.2	**37.8**	3.6	39.0	3.0	**32.3**	4.0
	16	54.2	3.1	27.0	6.3	9.1	9.2	18.1	4.8	36.7	3.6	**39.6**	3.3	30.8	5.0
7B scale															
Qwen2.5 SFT	–	73.8	1.0	59.2	1.0	41.6	1.0	27.9	1.0	69.1	1.0	**78.7**	1.0	58.4	1.0
Lafa (Ours)	4	**78.9**	1.7	**63.0**	3.4	**41.8**	2.4	**31.0**	2.3	73.8	2.8	76.8	2.8	**60.9**	2.6
	8	77.5	2.8	59.1	6.3	39.4	3.7	28.1	4.0	**75.3**	4.8	76.2	4.6	59.3	4.3
	16	74.9	2.8	57.3	7.9	39.0	3.8	28.1	4.0	75.0	5.8	73.8	5.4	58.0	4.9
8B scale															
Qwen3 SFT	–	38.1	1.0	**71.6**	1.0	**56.2**	1.0	29.5	1.0	62.9	1.0	**82.3**	1.0	56.8	1.0
Lafa (Ours)	4	**62.3**	2.4	70.7	3.4	**56.2**	3.2	**31.7**	3.1	**77.8**	2.6	79.9	2.9	**63.1**	2.9

3.2 Autoregressive decoding

We next examine how joint flow training affects the retained AR capability (Section 2.2), evaluating Lafa and baselines under single-token decoding. As shown in Table 1, Lafa improves average accuracy over matched AR SFT across all three model scales (per-benchmark results in Appendix C.1). It also outperforms the matched multi-token and block-parallel baselines on average at 0.5B and 7B. At 8B, Lafa achieves similar accuracy to Efficient-DLM evaluated with single-token unmasking, while using at least 18× fewer training tokens. These results show that learning the flow can also improve AR prediction, consistent with the benefits of multi-token training [6, 8, 34]. We therefore use this retained AR predictor to verify flow-based drafts for self-speculative decoding (Section 3.3).

3.3 Self-speculative decoding

We now evaluate self-speculative decoding, which verifies one-step flow drafts with the retained AR predictor (Section 2.3). We measure both parallelism, in tokens committed per forward pass (TPF), and end-to-end wall-clock throughput. Each block size uses its corresponding trained checkpoint, and self-speculation preserves that checkpoint's greedy AR output.

Modified document copy; original authors and local edits are listed on the cover.

Parallelism. Figure 7 and Table 2 report accuracy and TPF averaged over all six benchmarks (protocols detailed in Appendix C.1). At 0.5B, Lafa with block size eight achieves about twice the TPF of DeepSeek-MTP, MARS, and PTP while attaining higher average accuracy. At 7B, increasing the block size from four to sixteen nearly doubles TPF while keeping accuracy close to AR SFT. With block size four, Lafa exceeds DeepSeek-MTP in both accuracy and TPF; with block size eight, it matches the accuracy of MARS with block size four at about 1.5× its TPF. At 8B, Lafa achieves similar accuracy and TPF to Efficient-DLM despite using at least 18× fewer training tokens. Across scales, Lafa achieves approximately 2.5–5 TPF while remaining close to or above AR SFT accuracy.

Wall-clock throughput. Since forward-pass costs differ across methods, we also measure end-to-end throughput at 7–8B scales on a single H100 GPU with batch size one, using each method's native decoder. Figure 8 reports accuracy and throughput in tokens per second (TPS), averaged over IFEval, GSM8K, and HumanEval (settings and per-benchmark results in Appendix C.2). With block size sixteen, Lafa 7B achieves 2.54× the throughput of 7B AR SFT at comparable average accuracy. With block size four, it is about twice as fast while also more accurate on average. DeepSeek-MTP and MARS match or exceed AR SFT accuracy but achieve at most 1.4× its throughput. Diffusion baselines fall below 7B AR SFT accuracy, faster settings showing larger losses. Thus, while flow decoding requires multiple integration steps to reach AR quality (Section 3.1), verifying one-step flow drafts enables substantially faster generation at comparable quality.

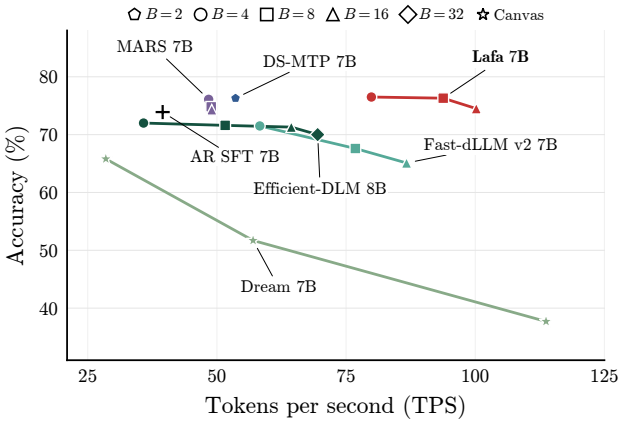


Figure 8: Accuracy ↑ vs. end-to-end throughput measured in tokens per second (TPS) ↑ at 7–8B scales, averaged over IFEval, GSM8K, and HumanEval.

Accepting drafts by confidence thresholding instead of AR verification trades accuracy for speed. We therefore use self-speculation as our default fast decoding mode and report confidence decoding results in Appendix C.3.

3.4 Ablations

We ablate the main training and inference choices in Section 2 using flow decoding to assess their effect on the learned flow. All comparisons use 0.5B models with block size four and 32 Euler integration steps (Table 3). For training, block-bidirectional attention and learned tokenwise time both improve accuracy over their causal and synchronous counterparts. Learning embedding directions while fixing their norms outperforms both frozen embeddings and unconstrained updates. Combining denoising cross entropy and embedding squared error improves accuracy over either objective alone. Adding prefix AR loss improves flow accuracy further, showing that retaining AR capabilities need not compromise flow generation. For inference, both self-conditioning and prefix guidance improve accuracy. Self-conditioning provides the larger gain while requiring no additional model evaluations during decoding. These results motivate better incorporation of both components into the flow map.

Table 3: Ablations. We use flow decoding and report mean accuracy over IFEval, GSM8K, and HumanEval.

Configuration	Average (%)
Training	
Fully causal attention	32.7
Block-bidirectional attention	**38.0**
Synchronous time	36.2
Asynchronous tokenwise time	**38.0**
Frozen embeddings	34.4
Unconstrained training	36.9
Norm-frozen training	**38.0**
Denoising CE	33.6
Denoising MSE	29.7
Denoising CE & MSE	35.8
Denoising CE & MSE + AR	**38.0**
Inference	
No self-conditioning	32.3
Self-conditioning	**38.1**
No guidance	34.3
Prefix guidance	**38.1**

Modified document copy; original authors and local edits are listed on the cover.

4 Related work

In this section, we review existing representative approaches for parallel language modeling.

Block-autoregressive language models. Block-autoregressive models reduce sequential generation by producing blocks of tokens conditioned on preceding blocks. Approaches to parallel block generation include multi-token prediction (MTP) heads or modules [6, 8], and predictors that take masked or noisy blocks as input [23, 33, 46]. Many parallel decoders predict tokens independently within an update, which can degrade quality when many tokens are committed at once [13]. Some methods mitigate this through AR verification or confidence-based acceptance of a draft prefix [8, 33, 41]. Some recent methods learn deterministic noise-to-token maps that can capture joint dependencies without independent output sampling [47, 57].

Discrete diffusion language models. Discrete diffusion models generate text by reversing a discrete corruption process, such as masking or random token replacement, through iterative denoising [58–61]. As with other parallel decoders, independent sampling within each update can limit quality in few-step generation [62, 63]. These models have been scaled to billions of parameters [11, 20, 21, 45], with generation performed over entire sequences or individual blocks. Block diffusion [23] combines iterative denoising within each block with autoregressive generation and prefix caching across blocks.

Continuous diffusion and flow language models. Continuous diffusion and flow models generate text through noisy continuous representations, including token embeddings, simplex representations, and binary encodings [12, 18, 28, 37, 64–67]. Continuous flows transport joint sequence representations and admit flow maps that can be learned for few-step generation [13, 38]. Recent work has demonstrated competitive generation quality and improved few-step decoding [13–17, 36, 68], while a few recent efforts have explored billion-parameter scales [69–72]. Related approaches also study autoregression and diffusion over autoencoded continuous representations [73–76]. Realizing the benefits of continuous flows at large and practical scale calls for data-efficient training and fast inference. Lafa addresses both by aligning blockwise asynchronous flows with the causal structure of pretrained AR models. This enables efficient adaptation and prefix caching, while retained AR prediction supports self-speculative generation.

5 Conclusion

We introduced Lafa, a blockwise flow language model scaled to 8B parameters by adapting pretrained AR models with fewer than 30B training tokens. Lafa combines causal generation across blocks with learned asynchronous flows within each block, preserving prefix caching while enabling parallel generation. With sufficient integration steps, flow decoding approaches or surpasses AR SFT accuracy, and flow map distillation improves few-step generation. Joint training also improves AR accuracy over matched AR SFT. This retained AR predictor verifies one-step flow drafts within the same model, enabling self-speculative decoding. With this approach, Lafa 7B achieves 2.54× the throughput of the 7B AR SFT baseline at comparable average accuracy. These results demonstrate a practical path to scaling continuous flows for language modeling through data-efficient AR adaptation. Improving flow map distillation, particularly its incorporation of self-conditioning and prefix guidance, remains a promising direction for further accelerating flow generation.

Acknowledgments

The authors would like to thank Sheel Shah, Manan Agarwal, Nick Boffi, Sander Dieleman, Jiaxin Shi, and Kyunghyun Cho for valuable discussions.

Modified document copy; original authors and local edits are listed on the cover.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. (page 1)

[2] Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023. (page 1)

[3] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025. (page 1)

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. (page 1)

[5] Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2):270–280, 1989. (page 1)

[6] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024. (pages 1, 4, 7, 8, 10, and 24)

[7] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024. (page 1)

[8] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024. (pages 1, 4, 7, 8, 10, 24, and 26)

[9] Inception Labs, Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, et al. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025. (page 1)

[10] Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuehang Yang, Hongli Yu, Xingwei Qu, et al. Seed diffusion: A large-scale diffusion language model with high-speed inference. *arXiv preprint arXiv:2508.02193*, 2025. (page 1)

[11] DiffusionGemma Team, Adrien Ali Taïga, James Assiene, Daniele Calandriello, Rahma Chaabouni, João Gante, Tamara von Glehn, Nate Keating, Chris Knutsen, Martin Kukla, et al. Diffusiongemma technical report. *arXiv preprint arXiv:2608.00146*, 2026. (pages 1 and 10)

[12] Sander Dieleman, Laurent Sartran, Arman Roshannai, Nikolay Savinov, Yaroslav Ganin, Pierre H Richemond, Arnaud Doucet, Robin Strudel, Chris Dyer, Conor Durkan, et al. Continuous diffusion for categorical data. *arXiv preprint arXiv:2211.15089*, 2022. (pages 1, 3, 4, 5, 10, and 20)

[13] Chanhyuk Lee, Jaehoon Yoo, Manan Agarwal, Sheel Shah, Jerry Huang, Aditi Raghunathan, Seunghoon Hong, Nicholas M Boffi, and Jinwoo Kim. Flow map language models: One-step language modeling via continuous denoising. *arXiv preprint arXiv:2602.16813*, 2026. (pages 1, 3, 4, 5, 10, 18, and 19)

[14] Keya Hu, Linlu Qiu, Yiyang Lu, Hanhong Zhao, Tianhong Li, Yoon Kim, Jacob Andreas, and Kaiming He. Elf: Embedded language flows. *arXiv preprint arXiv:2605.10938*, 2026. (pages 1, 5, and 10)

Modified document copy; original authors and local edits are listed on the cover.

[15] Yuxin Chen, Chumeng Liang, Hangke Sui, Ruihan Guo, Chaoran Cheng, Jiaxuan You, and Ge Liu. Langflow: Continuous diffusion rivals discrete in language modeling. *arXiv preprint arXiv:2604.11748*, 2026. (pages 1, 5, 10, and 20)

[16] Georgios Batzolis, Mark Girolami, and Luca Ambrogioni. Cobit: Language modeling with bitstream diffusion. *arXiv preprint arXiv:2605.07013*, 2026. (pages 1 and 10)

[17] Zhihan Yang, Wei Guo, Shuibai Zhang, Subham Sekhar Sahoo, Yongxin Chen, Arash Vahdat, Morteza Mardani, and John Thickstun. Continuous diffusion scales competitively with discrete diffusion for language. *arXiv preprint arXiv:2605.18530*, 2026. (pages 1 and 10)

[18] Ishaan Gulrajani and Tatsunori B Hashimoto. Likelihood-based diffusion language models. *Advances in Neural Information Processing Systems*, 36:16693–16715, 2023. (pages 2 and 10)

[19] Vassilis Papadopoulos, Jérémie Wenger, and Clément Hongler. Arrows of time for large language models. *arXiv preprint arXiv:2401.17505*, 2024. (pages 2 and 3)

[20] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025. (pages 2, 6, 7, 10, 24, and 26)

[21] Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, Jingyu Liu, Chengyue Wu, Hao Zhang, Enze Xie, Song Han, et al. Efficient-dlm: From autoregressive to diffusion language models, and beyond in speed. *arXiv preprint arXiv:2512.14067*, 2025. (pages 2, 6, 7, 10, 24, and 26)

[22] Chunqi Wang, Ji Zhang, and Haiqing Chen. Semi-autoregressive neural machine translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 479–488, 2018. (page 3)

[23] Marianne Arriola, Aaron Gokaslan, Justin Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*, volume 2025, pages 50726–50753, 2025. (pages 3, 4, 7, 10, 19, and 24)

[24] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. (page 3)

[25] Michael Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *Journal of Machine Learning Research*, 26(209):1–80, 2025. (pages 3 and 17)

[26] Alan Baade, Eric Ryan Chan, Kyle Sargent, Changan Chen, Justin Johnson, Ehsan Adeli, and Li Fei-Fei. Latent forcing: Reordering the diffusion trajectory for pixel-space image generation. *arXiv preprint arXiv:2602.11401*, 2026. (page 3)

[27] Bingshuo Qian and Xiang Cheng. Learning when to denoise: Optimizing asynchronous schedules for latent diffusion. *arXiv preprint arXiv:2606.19662*, 2026. (pages 3 and 17)

[28] Tong Wu, Zhihao Fan, Xiao Liu, Hai-Tao Zheng, Yeyun Gong, Jian Jiao, Juntao Li, Jian Guo, Nan Duan, Weizhu Chen, et al. Ar-diffusion: Auto-regressive diffusion model for text generation. *Advances in Neural Information Processing Systems*, 36:39957–39974, 2023. (pages 3 and 10)

[29] Parikshit Bansal and Sujay Sanghavi. Token time continuous diffusion for language modeling. *arXiv preprint arXiv:2607.14106*, 2026. (pages 3 and 17)

[30] Chanhyuk Lee, Wonjung Kim, Patrick Pynadath, Jaehoon Yoo, Satoshi Hayakawa, Ruqi Zhang, Jinwoo Kim, and Seunghoon Hong. Asynchronous flow maps for flow language models, 2026. Forthcoming. (pages 3 and 17)

Modified document copy; original authors and local edits are listed on the cover.

[31] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4172–4182. IEEE, 2023. (page 4)

[32] Yuchuan Tian, Yuchen Liang, Shuo Zhang, Yingte Shu, Guangwen Yang, Wei He, Sibo Fang, Tianyu Guo, Kai Han, Chao Xu, Hanting Chen, Xinghao Chen, and Yunhe Wang. From next-token to next-block: A principled adaptation path for diffusion LLMs. *arXiv preprint arXiv:2512.06776*, 2025. (pages 4 and 19)

[33] Ziqi Jin, Lei Wang, Ziwei Luo, and Aixin Sun. Mars: Enabling autoregressive models multi-token generation. *arXiv preprint arXiv:2604.07023*, 2026. (pages 4, 6, 7, 10, 19, 24, and 26)

[34] Divyat Mahajan, Sachin Goyal, Badr Youbi Idrissi, Mohammad Pezeshki, Ioannis Mitliagkas, David Lopez-Paz, and Kartik Ahuja. Beyond multi-token prediction: Pretraining llms with future summaries. In *International Conference on Learning Representations*, volume 2026, pages 64342–64364, 2026. (pages 4 and 8)

[35] Aiwei Liu, Minghua He, Shaoxun Zeng, Sijun Zhang, Linhao Zhang, Chuhan Wu, Wei Jia, Yuan Liu, Xiao Zhou, and Jie Zhou. Wedlm: Reconciling diffusion language models with standard causal attention for fast inference. *arXiv preprint arXiv:2512.22737*, 2025. (pages 4 and 19)

[36] Jaehoon Yoo, Wonjung Kim, Floor Eijkelboom, Chanhyuk Lee, Nicholas M Boffi, Seunghoon Hong, and Jinwoo Kim. Self-conditioned flow map language models via fixed-point flows. *arXiv preprint arXiv:2607.00714*, 2026. (pages 5 and 10)

[37] Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. Analog bits: Generating discrete data using diffusion models with self-conditioning. *arXiv preprint arXiv:2208.04202*, 2022. (pages 5 and 10)

[38] Nicholas Boffi, Michael Albergo, and Eric Vanden-Eijnden. How to build a consistency model: Learning flow maps via self-distillation. *Advances in Neural Information Processing Systems*, 38:33346–33382, 2026. (pages 5, 10, and 18)

[39] Kiwhan Song, Boyuan Chen, Max Simchowitz, Yilun Du, Russ Tedrake, and Vincent Sitzmann. History-guided video diffusion. *arXiv preprint arXiv:2502.06764*, 2025. (pages 6 and 21)

[40] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. (page 6)

[41] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31, 2018. (pages 6 and 10)

[42] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. (page 6)

[43] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. (page 6)

[44] Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, et al. Olmo 3. *arXiv preprint arXiv:2512.13961*, 2025. (page 6)

Modified document copy; original authors and local edits are listed on the cover.

[45] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *Advances in Neural Information Processing Systems*, 38:50608–50646, 2026. (pages 6, 7, 10, and 24)

[46] Itai Gat, Heli Ben-Hamu, Marton Havasi, Daniel Haziza, Jeremy Reizenstein, Gabriel Synnaeve, David Lopez-Paz, Brian Karrer, and Yaron Lipman. Set block decoding is a language model inference accelerator. *arXiv preprint arXiv:2509.04185*, 2025. (pages 7, 10, and 24)

[47] Felix Draxler, Justus Will, Farrin Sofian, Theofanis Karaletsos, Sameer Singh, and Stephan Mandt. Parallel token prediction for language models. In *International Conference on Learning Representations*, volume 2026, pages 5624–5652, 2026. (pages 7 and 10)

[48] Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dllm v2: Efficient block-diffusion llm. In *International Conference on Learning Representations*, volume 2026, pages 128353–128370, 2026. (pages 7, 24, and 26)

[49] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023. (page 7)

[50] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed H Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, 2023. (page 7)

[51] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024. (page 7)

[52] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023. (page 7)

[53] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. (page 7)

[54] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. (page 7)

[55] Antonio Franca and Alexander Tong. Hacking generative perplexity: Why unconditional text evaluation needs distributional metrics. *arXiv preprint arXiv:2606.08417*, 2026. (page 7)

[56] Lintang Sutawika, Hailey Schoelkopf, Leo Gao, Baber Abbasi, Stella Biderman, Jonathan Tow, et al. EleutherAI/lm-evaluation-harness: v0.4.3, 07 2024. URL https://zenodo.org/records/12608602. (page 7)

[57] Chi Zhang, Xixi Hu, Bo Liu, and Qiang Liu. Gumbel distillation for parallel text generation. *arXiv preprint arXiv:2603.22216*, 2026. (page 10)

[58] Subham S Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024. (page 10)

Modified document copy; original authors and local edits are listed on the cover.

[59] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. *Advances in neural information processing systems*, 37:103131–103167, 2024. (page 10)

[60] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023. (page 10)

[61] Yair Schiff, Subham Sahoo, Hao Phung, Guanghan Wang, Sam Boshar, Hugo Dalla-Torre, Bernardo Almeida, Alexander Rush, Thomas Pierrot, and Volodymyr Kuleshov. Simple guidance mechanisms for discrete diffusion models. In *International Conference on Learning Representations*, volume 2025, pages 43776–43821, 2025. (page 10)

[62] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34:17981–17993, 2021. (page 10)

[63] Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky TQ Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. *Advances in Neural Information Processing Systems*, 37:133345–133385, 2024. (page 10)

[64] Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. *Advances in neural information processing systems*, 35:4328–4343, 2022. (page 10)

[65] Xiaochuang Han, Sachin Kumar, and Yulia Tsvetkov. Ssd-lm: Semi-autoregressive simplex-based diffusion language model for text generation and modular control. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11575–11596, 2023. (page 10)

[66] Robin Strudel, Corentin Tallec, Florent Altché, Yilun Du, Yaroslav Ganin, Arthur Mensch, Will Grathwohl, Nikolay Savinov, Sander Dieleman, Laurent Sifre, et al. Self-conditioned embedding diffusion for text generation. *arXiv preprint arXiv:2211.04236*, 2022. (page 10)

[67] Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Shekhtman, and Kilian Q Weinberger. Latent diffusion for language generation. *Advances in Neural Information Processing Systems*, 36:56998–57025, 2023. (page 10)

[68] Manan Agarwal, Sheel Shah, Chanhyuk Lee, Jaehoon Yoo, Jerry Huang, Seunghoon Hong, Aditi Raghunathan, Jinwoo Kim, and Nicholas M Boffi. Posterior refinement: Fast language generation via any-order flow maps. *arXiv preprint arXiv:2606.24773*, 2026. (page 10)

[69] Jaesung Tae, Hamish Ivison, Sachin Kumar, and Arman Cohan. Tess 2: A large-scale generalist diffusion language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21171–21188, 2025. (page 10)

[70] Oscar Davis, Anastasiia Filippova, Pierre Ablin, Victor Turrisi, Amitis Shidani, Marco Cuturi, and Louis Béthune. Scaling categorical flow maps. *arXiv preprint arXiv:2605.07820*, 2026. (page 10)

[71] Shaobin Zhuang, Yuang Ai, Jiaming Han, Xiaohui Li, Huaibo Huang, Xiangyu Yue, Xuefeng Hu, Kun Xu, Yali Wang, and Hao Chen. Bitlm: Unlocking multi-token language generation with bitwise continuous diffusion. *arXiv preprint arXiv:2605.11577*, 2026. (page 10)

[72] Longxuan Yu, Yunshu Wu, Yu Fu, Siheng Xiong, Rob Brekelmans, Hui Liu, Yue Dong, and Greg Ver Steeg. Dsl-llada: Scaling continuous denoising to 8b masked diffusion lms. *arXiv preprint arXiv:2606.01024*, 2026. (page 10)

Modified document copy; original authors and local edits are listed on the cover.

[73] Chenze Shao, Darren Li, Fandong Meng, and Jie Zhou. Continuous autoregressive language models. *arXiv preprint arXiv:2510.27688*, 2025. (page 10)

[74] Hongcan Guo, Qinyu Zhao, Yian Zhao, Shen Nie, Rui Zhu, Qiushan Guo, Feng Wang, Tao Yang, Hengshuang Zhao, Guoqiang Wei, et al. Continuous latent diffusion language model. *arXiv preprint arXiv:2605.06548*, 2026. (page 10)

[75] Jiaxiu Jiang, Jingjing Ren, Wenbo Li, Bo Wang, Haoze Sun, Yijun Yang, Jianhui Liu, Yanbing Zhang, Shenghe Zheng, Yuan Zhang, et al. Textldm: Language modeling with continuous latent diffusion. *arXiv preprint arXiv:2605.07748*, 2026. (page 10)

[76] Jiajun Liang, Yucheng Liao, Yukang Cao, Jiazhe Wei, Ken Li, Wende Tan, Jiankun Zhang, ZY Cui, Jingkang Yang, Liucheng Guo, et al. Aurora-lm: Autoencoding unified representation for continuous-latent diffusion language modeling. *arXiv preprint arXiv:2608.02602*, 2026. (page 10)

[77] Yiyang Lu, Susie Lu, Qiao Sun, Hanhong Zhao, Zhicheng Jiang, Xianbang Wang, Tianhong Li, Zhengyang Geng, and Kaiming He. One-step latent-free image generation with pixel mean flows. *arXiv preprint arXiv:2601.22158*, 2026. (page 18)

Modified document copy; original authors and local edits are listed on the cover.

A Theoretical results

A.1 Asynchronous flow velocity and denoiser

In this subsection, we derive the relationship between the velocity field and the denoiser of the asynchronous flow (5) used in Section 2.1.

Proposition A.1. *For the interpolant (2), the velocity* $b_t(\mathbf{x};\mathbf{c}) = \mathbb{E}[\dot{I}_t \mid \mathbf{c}, I_t = \mathbf{x}]$ *satisfies, at each position* i*,*

$$b_t(\mathbf{x};\mathbf{c})^i = \frac{\dot{\tau}^i(t)}{1-\tau^i(t)}\big(D_t(\mathbf{x};\mathbf{c})^i\mathbf{E} - \mathbf{x}^i\big), \qquad t \in [0,1).$$

Proof. The time derivative of the interpolant is

$$\dot{I}_t^i = \dot{\tau}^i(t)(\mathbf{x}_1^i - \mathbf{x}_0^i), \tag{11}$$

and its conditional expectation gives the velocity [25, 27]. Solving (2) for the noise gives

$$\mathbf{x}_0^i = \frac{I_t^i - \tau^i(t)\mathbf{x}_1^i}{1-\tau^i(t)}, \qquad \text{so} \qquad \mathbf{x}_1^i - \mathbf{x}_0^i = \frac{\mathbf{x}_1^i - I_t^i}{1-\tau^i(t)}.$$

Substituting into $b_t(\mathbf{x};\mathbf{c}) = \mathbb{E}[\dot{I}_t \mid \mathbf{c}, I_t = \mathbf{x}]$ with $\dot{I}_t^i = \dot{\tau}^i(t)(\mathbf{x}_1^i - \mathbf{x}_0^i)$,

$$b_t(\mathbf{x};\mathbf{c})^i = \frac{\dot{\tau}^i(t)}{1-\tau^i(t)}\,\mathbb{E}\big[\mathbf{x}_1^i - I_t^i \mid \mathbf{c}, I_t = \mathbf{x}\big] = \frac{\dot{\tau}^i(t)}{1-\tau^i(t)}\Big(\mathbb{E}\big[\mathbf{x}_1^i \mid \mathbf{c}, I_t = \mathbf{x}\big] - \mathbf{x}^i\Big).$$

Since $\mathbf{x}_1^i = \mathbf{E}_{\mathbf{y}^i}$, the conditional mean is the posterior mean embedding under (4),

$$\mathbb{E}[\mathbf{x}_1^i \mid \mathbf{c}, I_t = \mathbf{x}] = \sum_{v \in V} p(\mathbf{y}^i = v \mid \mathbf{c}, I_t = \mathbf{x})\,\mathbf{E}_v = D_t(\mathbf{x};\mathbf{c})^i\mathbf{E}. \tag{12}$$

□

A.2 Asynchronous embedded flow maps

In this subsection, we derive the embedded flow map used in Section 2.2, first characterizing the two-time denoiser (7) and then giving the full distillation objective (20).

Setup. Throughout, the prefix $\mathbf{c}$ is fixed, and the vocabulary embeddings $\mathbf{E} \in \mathbb{R}^{|V|\times d}$ and tokenwise schedules are held fixed. A block has positions $i = 1, ..., B$ with clean embeddings $\mathbf{x}_1^i = \mathbf{E}_{\mathbf{y}^i}$. For each position i, let τ^i be continuously differentiable with $\dot{\tau}^i(t) > 0$ for $t < 1$, $\tau^i(0) = 0$, and $\tau^i(1) = 1$, and write $\tau_t^i \coloneqq \tau^i(t)$. Since all tokenwise times are driven by the single global time t, they trace a one-dimensional monotone curve in $[0,1]^B$, and the flow map is indexed by two global times [29, 30]. Let $\hat{D}_t$ be the frozen teacher denoiser (Section 2.2). As in (5), it induces the velocity and flow map

$$\begin{aligned} b_t(\mathbf{x};\mathbf{c})^i &= \frac{\dot{\tau}^i(t)}{1-\tau_t^i}\big(\hat{D}_t(\mathbf{x};\mathbf{c})^i\mathbf{E} - \mathbf{x}^i\big), \\ \partial_t X_{s,t}(\mathbf{x};\mathbf{c}) &= b_t(X_{s,t}(\mathbf{x};\mathbf{c});\mathbf{c}), \qquad X_{s,s}(\mathbf{x};\mathbf{c}) = \mathbf{x}. \end{aligned} \tag{13}$$

We work on $0 \le s \le t < 1$ and assume unique ODE solutions and that all denoisers are continuous and differentiable in time. Expressions at the endpoint $t = 1$, used in flow map inference, are understood as limits when they exist.

Modified document copy; original authors and local edits are listed on the cover.

Two-time denoiser. For $0 \le s < t < 1$, we define the average velocity $v_{s,t}$ between s and t as [38]

$$v_{s,t}(\mathbf{x}; \mathbf{c}) \coloneqq \frac{X_{s,t}(\mathbf{x}; \mathbf{c}) - \mathbf{x}}{t - s}. \tag{14}$$

To express transport between two times through a denoising prediction, we define a two-time denoiser [13, 77] $\delta_{s,t}(\mathbf{x}; \mathbf{c}) \in (\Delta^{|V|-1})^B$ through its mean embedding,

$$\delta_{s,t}(\mathbf{x}; \mathbf{c})^i \mathbf{E} = \mathbf{x}^i + \frac{(1 - \tau_s^i)(t - s)}{\tau_t^i - \tau_s^i} v_{s,t}(\mathbf{x}; \mathbf{c})^i. \tag{15}$$

Substituting (14) and rearranging gives (7). Since $|V| \gg d$, distinct probability vectors can share the same mean embedding. Thus (15) fixes only the mean embedding $\delta_{s,t}\mathbf{E}$ and not δ itself. The following proposition constructs a specific δ and identifies the conditions under which a learned denoiser recovers the flow.

Proposition A.2. *There exists a two-time denoiser δ satisfying (15) with the following two conditions.*

1. *(Diagonal condition) $\delta_{s,t} \to \hat{D}_s$ as $t \to s$, so that $\delta_{s,s} \coloneqq \hat{D}_s$.*
2. *(Semigroup condition) For $s < u < t$,*

$$\delta_{s,t}(\mathbf{x}; \mathbf{c})^i = \gamma^i \delta_{s,u}(\mathbf{x}; \mathbf{c})^i + (1 - \gamma^i)\delta_{u,t}(X_{s,u}(\mathbf{x}; \mathbf{c}); \mathbf{c})^i, \qquad \gamma^i = \frac{(1 - \tau_t^i)(\tau_u^i - \tau_s^i)}{(1 - \tau_u^i)(\tau_t^i - \tau_s^i)}. \tag{16}$$

Conversely, for any two-time denoiser $\delta_{s,t}$ satisfying the diagonal and semigroup conditions, its associated flow map defined by (7) equals the true flow map $X_{s,t}$.

Proof. By the velocity in the setup,

$$\frac{\mathrm{d}}{\mathrm{d}r} \frac{X_{s,r}^i}{1 - \tau_r^i} = \frac{(1 - \tau_r^i)\, \partial_r X_{s,r}^i + \dot{\tau}^i(r)\, X_{s,r}^i}{(1 - \tau_r^i)^2} = w^i(r)\, \hat{D}_r(X_{s,r})^i \mathbf{E}, \qquad w^i(r) \coloneqq \frac{\dot{\tau}^i(r)}{(1 - \tau_r^i)^2}, \tag{17}$$

where we suppress $\mathbf{x}$ and $\mathbf{c}$. Define

$$\delta_{s,t}^i \coloneqq \frac{1}{Z_{s,t}^i} \int_s^t w^i(r)\, \hat{D}_r(X_{s,r})^i \, \mathrm{d}r, \qquad Z_{s,t}^i \coloneqq \int_s^t w^i(r)\, \mathrm{d}r = \frac{\tau_t^i - \tau_s^i}{(1 - \tau_t^i)(1 - \tau_s^i)}. \tag{18}$$

This is a weighted average of probability vectors, hence a probability vector.

Integrating (17) over $[s, t]$ and multiplying by $1 - \tau_t^i$ gives $X_{s,t}^i = \frac{1-\tau_t^i}{1-\tau_s^i}\mathbf{x}^i + \frac{\tau_t^i - \tau_s^i}{1-\tau_s^i}\delta_{s,t}^i \mathbf{E}$, which is (15).

Diagonal condition. As $t \to s$, the normalized weights $w^i / Z_{s,t}^i$ concentrate at $r = s$, so $\delta_{s,t} \to \hat{D}_s$ by continuity of the teacher along the trajectory.

Semigroup condition. Split the integral at u. The part over $[s, u]$ is $Z_{s,u}^i \delta_{s,u}^i$. For $r \ge u$, $X_{s,r} = X_{u,r}(X_{s,u})$, so the part over $[u, t]$ is $Z_{u,t}^i \delta_{u,t}(X_{s,u})^i$. Dividing by $Z_{s,t}^i = Z_{s,u}^i + Z_{u,t}^i$ gives (16) with $\gamma^i = Z_{s,u}^i / Z_{s,t}^i$.

Converse. Let $\hat{X}$ be defined from δ by (7). Then $\hat{X}$ is the flow map of b_t if $\hat{X}_{s,s} = \mathrm{Id}$ and it satisfies the tangent condition $\partial_t \hat{X}_{s,t}|_{t=s} = b_s$ and the semigroup condition $\hat{X}_{s,t} = \hat{X}_{u,t} \circ \hat{X}_{s,u}$ [38]. We verify each. $\hat{X}_{s,s} = \mathrm{Id}$ holds by construction. The tangent condition follows from the diagonal condition by differentiating (7) at $t = s$. Substituting (7) twice, $\hat{X}_{u,t} \circ \hat{X}_{s,u}$ has the form of (7) with denoiser $\gamma^i \delta_{s,u}^i + (1 - \gamma^i)\delta_{u,t}(\hat{X}_{s,u})^i$, so the semigroup condition on δ gives the semigroup condition on $\hat{X}$. Hence $\hat{X} = X$. □

Modified document copy; original authors and local edits are listed on the cover.

Distillation objective. By Theorem A.2, learning the flow map reduces to satisfying the diagonal and semigroup conditions. We impose both with KL losses on a student $\hat{\delta}_{s,t}$ initialized from the frozen teacher $\hat{D}_s$ (Section 2.2).

For $0 \le s < u < t < 1$, the off-diagonal target composes two shorter student transitions [13],

$$\bar{\delta}_{s,t}(\mathbf{x}_s; \mathbf{c})^i \coloneqq \mathsf{sg}\Big[\gamma^i \, \hat{\delta}_{s,u}(\mathbf{x}_s; \mathbf{c})^i + (1-\gamma^i)\, \hat{\delta}_{u,t}(\mathbf{x}_u; \mathbf{c})^i\Big], \qquad \mathbf{x}_u \coloneqq \hat{X}_{s,u}(\mathbf{x}_s; \mathbf{c}), \tag{19}$$

with γ^i as in (16). Since $0 \le \gamma^i \le 1$, the target is a categorical distribution. The intermediate state $\mathbf{x}_u$ is computed by the student, and sg stops gradients through the entire target. Since prefix positions are conditioned on time 1, the student retains the prefix cache and an autoregressive predictor $f^{\hat{\delta}}$. We train the student with semigroup-based flow map distillation [13] plus autoregressive distillation:

$$\mathcal{L}_{\mathsf{distill}}(\hat{\delta}) \coloneqq \underbrace{\mathbb{E}_{\mathbf{c}}\, \mathsf{KL}(f(\mathbf{c}) \| f^{\hat{\delta}}(\mathbf{c}))}_{\text{AR distillation loss}} + \underbrace{\mathbb{E}_{t,\mathbf{c},I_t}\, \mathsf{KL}(\hat{D}_t(I_t;\mathbf{c}) \| \hat{\delta}_{t,t}(I_t;\mathbf{c}))}_{\text{diagonal distillation loss}} + \underbrace{\mathbb{E}_{s,u,t,\mathbf{c},I_s}\, \mathsf{KL}(\bar{\delta}_{s,t}(I_s;\mathbf{c}) \| \hat{\delta}_{s,t}(I_s;\mathbf{c}))}_{\text{off-diagonal distillation loss}}, \tag{20}$$

where each KL sums tokenwise over prefix or block positions. The first two loss terms match the student's AR predictions and instantaneous denoising with the teacher, while the off-diagonal term enforces the semigroup property $\hat{X}_{s,t} \approx \hat{X}_{u,t} \circ \hat{X}_{s,u}$, bootstrapping long jumps from short ones up to one-step generation $\hat{X}_{0,1}$.

The following proposition shows that the two flow map terms of this objective are well posed, in that their population minimizers recover the teacher flow map.

Proposition A.3. *Assume full support of the sampling distributions over (s,u,t) and the states. Over all two-time denoisers δ, the population objective*

$$\mathcal{L}(\delta) = \mathbb{E}_{t,I_t} \sum_{i=1}^{B} \mathsf{KL}\big(\hat{D}_t(I_t;\mathbf{c})^i \,\|\, \delta_{t,t}(I_t;\mathbf{c})^i\big) + \mathbb{E}_{s,u,t,I_s} \sum_{i=1}^{B} \mathsf{KL}\big(\bar{\delta}_{s,t}(I_s;\mathbf{c})^i \,\|\, \delta_{s,t}(I_s;\mathbf{c})^i\big) \tag{21}$$

attains its minimum zero, and every population minimizer recovers the teacher flow map X through (7).

Proof. The denoiser constructed in Theorem A.2 satisfies both conditions, so both KL terms are zero. Conversely, a zero loss implies the diagonal and semigroup conditions almost surely, and by full support and continuity, everywhere. By the converse of Theorem A.2, $\hat{X} = X$. □

Note that the two terms of (21) are the diagonal and off-diagonal terms of (20). The autoregressive term depends only on the prefix readout and is zero at $f = f^{\hat{\delta}}$, so it does not change the population minimizers of the two flow map terms.

B Algorithmic details

We provide training and inference pseudocode in Algorithms 1 to 4, and provide the hyperparameters used for our main results in Table 4.

B.1 Training

To train on many blocks at once, following prior work on block diffusion [23, 32, 33, 35], we pair each clean training sequence with a copy which is split into noisy blocks of size B. The attention mask applies prefix-causal and block-bidirectional structure to every block at once, such that clean positions attend causally, and each noisy block attends to all clean tokens preceding it and bidirectionally within itself, but not to later clean tokens or other noisy blocks. Under this setting, each step of training progresses as in Algorithm 1.

Modified document copy; original authors and local edits are listed on the cover.

Algorithm 1: A step of parallel denoiser training

Input: Training sequence $\mathbf{y} \in V^{R \times B}$, block size B, transformer f with embeddings $\mathbf{E}$, tokenwise time τ, noise scale σ, self-conditioning probability p_{sc}

$(\mathbf{y}^{(1)}, ..., \mathbf{y}^{(R)}) \leftarrow \mathbf{y}$ ▷ R blocks of B clean tokens
for *each block* r **do**
  $t_r \sim \mathsf{U}[0,1]$ ▷ uniform base time
  $\mathbf{x}_1^{(r)} \leftarrow \mathbf{E}_{\mathbf{y}^{(r)}}$ ▷ data embedding
  $\mathbf{x}_0^{(r)} \sim \mathsf{N}(0, \sigma^2 I)$ ▷ noise
  $I_{t_r}^{(r)} \leftarrow (1 - \tau(t_r))\mathbf{x}_0^{(r)} + \tau(t_r)\mathbf{x}_1^{(r)}$ ▷ interpolant (2)
$I \leftarrow (I_{t_1}^{(1)}, ..., I_{t_R}^{(R)})$ ▷ R blocks of B noisy tokens
$\boldsymbol{\tau} \leftarrow (\tau(t_1), ..., \tau(t_R))$ ▷ R blocks of B tokenwise times
if $\mathsf{U}[0,1] < p_{\mathsf{sc}}$ **then**
  Self-condition the noisy blocks following (24)
$(\mathbf{q}, \mathbf{p}) \leftarrow f_{(\mathbf{1}, \boldsymbol{\tau})}(\mathbf{y}, I)$ ▷ AR and denoising (Figure 4)
$\mathcal{L}(f) \leftarrow \mathsf{CE}(\mathbf{q}, \mathbf{y}) + \mathsf{CE}(\mathbf{p}, \mathbf{y}) + |\mathbf{pE} - \mathbf{E}_{\mathbf{y}}|^2$ ▷ loss (6)
Update f including $\mathbf{E}$ using $\mathcal{L}(f)$
Project each $\mathbf{E}_v$ to its pretrained norm
return f

Algorithm 2: A step of updating tokenwise time

Input: Denoiser D with embeddings $\mathbf{E}$, loss matrix $\mathbf{L} \in \mathbb{R}^{B \times M}$, block size B, time bins M, a batch of training sequences $\mathbf{y}$, noise scale σ, EMA decay β

$\bar{\mathbf{L}} \leftarrow \mathbf{0} \in \mathbb{R}^{B \times M}$ ▷ position-time bins
for *each sequence* $\mathbf{y}$ **do**
  Sample a block $\mathbf{y}^{(r)} \in V^B$ from $\mathbf{y}$
  $t_r \sim \mathsf{U}[0,1]$ ▷ uniform base time
  $\mathbf{x}_1^{(r)} \leftarrow \mathbf{E}_{\mathbf{y}^{(r)}}$ ▷ embedding
  $\mathbf{x}_0^{(r)} \sim \mathsf{N}(0, \sigma^2 I)$ ▷ noise
  $I_{t_r}^{(r)} \leftarrow (1 - t_r)\mathbf{x}_0^{(r)} + t_r \mathbf{x}_1^{(r)}$ ▷ synced interpolant
  $\mathbf{p}^{(r)} \leftarrow D_{t_r}(I_{t_r}^{(r)}; \mathbf{y}^{(1:r-1)})$ ▷ block denoising
  $\bar{\mathbf{L}} \leftarrow \mathsf{record}\big(\bar{\mathbf{L}}, \mathsf{CE}(\mathbf{p}^{(r)}, \mathbf{y}^{(r)}), t_r\big)$; ▷ running mean
$\mathbf{L}_{:,m} \leftarrow \beta \mathbf{L}_{:,m} + (1-\beta)\bar{\mathbf{L}}_{:,m}$ for each visited m ▷ EMA (23)
for *each position* $i = 1, ..., B$ **do**
  $F_i \leftarrow \mathsf{CDF}(\frac{\mathbf{L}_{i,:}}{\sum_m \mathbf{L}_{i,m}})$ ▷ piecewise linear CDF
  $\tau^i \leftarrow F_i^{-1}$ ▷ tokenwise time (22)
return $\mathbf{L}, \tau$

During training, inspired by the loss-aware time warping of Dieleman et al. [12], applied separately to each block position, we learn each tokenwise time as the inverse cumulative distribution of its denoising loss. Let $\ell_i(\tau) > 0$ be the expected denoising cross-entropy loss of position i at tokenwise time τ, and define

$$\tau^i(t) \coloneqq F_i^{-1}(t), \qquad F_i(\tau) \coloneqq \frac{\int_0^\tau \ell_i(r)\,\mathrm{d}r}{\int_0^1 \ell_i(r)\,\mathrm{d}r}. \tag{22}$$

Since $\dot{\tau}^i \propto 1/\ell_i(\tau^i)$ is small where the loss is high, both a uniform temporal grid over t during inference and uniform sampling of t during training concentrate on the hardest noise levels.

To estimate F during training, we keep an exponential moving average $\mathbf{L} \in \mathbb{R}^{B \times M}$ of the per-position denoising cross entropy in M uniform time bins, and set each F_i to the piecewise-linear CDF of the normalized row $\mathbf{L}_{i,:}$. We initialize $\mathbf{L}$ uniformly, and every $S \in \mathbb{N}$ optimization steps, run four gradient-free forward passes with independently sampled uniform times for each noisy block (see Algorithm 2 for details). For every position-time bin (i, m) encountered during these forward passes, we update

$$\mathbf{L}_{i,m} \leftarrow \beta \mathbf{L}_{i,m} + (1 - \beta)\bar{\mathbf{L}}_{i,m}, \tag{23}$$

where $\bar{\mathbf{L}}_{i,m}$ is the mean cross entropy recorded at position i in time bin m, and bins that are not visited retain their previous value. This independent uniform estimation prevents the learned sampling distribution from starving low-probability regions.

B.2 Flow decoding

To improve flow decoding, we apply self-conditioning during training with a fixed probability by obtaining an initial prediction without gradient and using its expected token embeddings to condition a second forward pass. Following Chen et al. [15], we incorporate this prediction through additional projection layers:

$$\bar{\mathbf{x}}_1 = \mathsf{sg}\big(D_t(I_t; \mathbf{c})\mathbf{E}\big), \qquad \tilde{I}_t = I_t + I_t \mathbf{W}_1^\top + \bar{\mathbf{x}}_1 \mathbf{W}_2^\top, \tag{24}$$

where $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times d}$ are trainable and initialized to zero. For training under self-conditioning, we replace the noisy block input with $\tilde{I}_t$ and compute the training loss on the second pass. Otherwise, we train with a single pass on I_t. In practice, we use a probability $p_{\mathsf{sc}} = 0.5$ of using self-conditioning for each training batch.

Modified document copy; original authors and local edits are listed on the cover.

Algorithm 3: Flow decoding

Input: Denoiser D with embeddings $\mathbf{E}$, prefix $\mathbf{c}$, tokenwise time τ, block size B, Euler steps K, guidance scale $g \geq 1$, noise scale σ

$\hat{\mathbf{x}}_0 \sim \mathsf{N}(0, \sigma^2 I)$ ▷ initial noise
$t_k \leftarrow k/K$ for $k = 0, \ldots, K$ ▷ base time grid
$\mathbf{s}, \mathbf{s}' \leftarrow \varnothing, \varnothing$ ▷ self-conditioning
for *each step* $k = 0, ..., K-1$ **do**
  $\mathbf{s} \leftarrow D_{t_k}(\mathsf{SC}(\hat{\mathbf{x}}_k, \mathbf{s}); \mathbf{c})$ ▷ usual forward
  if $g > 1$ **then**
    $\mathbf{s}' \leftarrow D_{t_k}(\mathsf{SC}(\hat{\mathbf{x}}_k, \mathbf{s}'); \varnothing)$ ▷ prefix-free forward
    $\tilde{\mathbf{x}}_k \leftarrow \mathbf{s}'\mathbf{E} + g(\mathbf{s}\mathbf{E} - \mathbf{s}'\mathbf{E})$ ▷ guided
  else
    $\tilde{\mathbf{x}}_k \leftarrow \mathbf{s}\mathbf{E}$ ▷ unguided
  for *each position* $i = 1, ..., B$ **do**
    $\hat{\mathbf{x}}^i_{k+1} \leftarrow \hat{\mathbf{x}}^i_k + \frac{\tau^i(t_{k+1}) - \tau^i(t_k)}{1 - \tau^i(t_k)}(\tilde{\mathbf{x}}^i_k - \hat{\mathbf{x}}^i_k)$ ▷ step
$\hat{\mathbf{y}} \leftarrow (\mathsf{argmax}(\mathbf{s}^j))_{j=1}^B$ ▷ decoding
return $\hat{\mathbf{y}}$

Algorithm 4: Fused self-speculative decoding

Input: Denoiser D and AR predictor f sharing a transformer, prefix $\mathbf{c}$ with KV cache $\mathbf{h}$, block size B, scale σ

$\mathbf{p} \leftarrow D_0(\mathbf{z}; \mathbf{c})$, $\mathbf{z} \sim \mathsf{N}(0, \sigma^2 I)$, $\mathbf{q} \leftarrow f(\mathbf{c})$ ▷ draft & AR prediction
while *the stopping condition is not met* **do**
  $\hat{\mathbf{y}} \leftarrow (\mathsf{argmax}(\mathbf{p}^i))_{i=1}^B$ ▷ rounding
  if $\hat{\mathbf{y}}^1 \neq \mathsf{argmax}(\mathbf{q}^{|c|})$ **then**
    $\hat{\mathbf{y}}^1 \leftarrow \mathsf{argmax}(\mathbf{q}^{|c|})$, $A \leftarrow \{1\}$ ▷ correction
  else
    $A \leftarrow [B]$ ▷ all possible acceptance lengths
  $\mathbf{z}^{(j)} \sim \mathsf{N}(0, \sigma^2 I)$ for $j \in A$ ▷ noise blocks
  Run a single forward pass on prefix $\mathbf{c} \oplus \hat{\mathbf{y}}^{1:\max A}$ and blocks $(\mathbf{z}^{(j)})_{j \in A}$ with KV cache $\mathbf{h}$ ▷ (Figure 9)
  $(\mathbf{q}; \mathbf{h}) \leftarrow f(\mathbf{c} \oplus \hat{\mathbf{y}}^{1:\max A})$ ▷ get AR prediction and KV
  $\mathbf{p}^{(j)} \leftarrow D_0(\mathbf{z}^{(j)}; \mathbf{c} \oplus \hat{\mathbf{y}}^{1:j})$ for $j \in A$ ▷ get drafts
  $a \leftarrow \max\{j \in A : \hat{\mathbf{y}}^{i+1} = \mathsf{argmax}(\mathbf{q}^{|c|+i})\ \forall 1 \leq i < j\}$
  $(\mathbf{c}, \mathbf{h}, \mathbf{q}, \mathbf{p}) \leftarrow (\mathbf{c} \oplus \hat{\mathbf{y}}^{1:a}, \mathbf{h}^{:|c|+a}, \mathbf{q}^{:|c|+a}, \mathbf{p}^{(a)})$ ▷ update
return $\mathbf{c}$

During inference, we use the denoiser prediction from the previous integration step for self-conditioning. In single-step update of Euler integration of (8), we replace $D_{t_k}(\hat{\mathbf{x}}_{t_k}; \mathbf{c})$ with $\mathbf{q}_k$, defined by

$$\tilde{\mathbf{x}}_{t_k} = \hat{\mathbf{x}}_{t_k} + \hat{\mathbf{x}}_{t_k}\mathbf{W}_1^\top + (\mathbf{q}_{k-1}\mathbf{E})\mathbf{W}_2^\top, \qquad \mathbf{q}_k = D_{t_k}(\tilde{\mathbf{x}}_{t_k}; \mathbf{c}), \qquad \mathbf{q}_0 = D_0(\hat{\mathbf{x}}_0; \mathbf{c}). \tag{25}$$

Detailed procedure for flow decoding can be found in Algorithm 3, where we write, for compactness:

$$\mathsf{SC}(\mathbf{x}, \mathbf{q}) \coloneqq \mathbf{x} + \mathbf{x}\mathbf{W}_1^\top + (\mathbf{q}\mathbf{E})\mathbf{W}_2^\top, \qquad \mathsf{SC}(\mathbf{x}, \varnothing) \coloneqq \mathbf{x}. \tag{26}$$

In addition to self-conditioning, during flow decoding, we employ prefix guidance to strengthen the conditioning on the prefix, analogously to history guidance in video diffusion models [39]. We define the guided denoiser as

$$D_t^g(\mathbf{x}; \mathbf{c}) \coloneqq D_t(\mathbf{x}; \varnothing) + g(D_t(\mathbf{x}; \mathbf{c}) - D_t(\mathbf{x}; \varnothing)), \qquad g \geq 1, \tag{27}$$

and replace $D_{t_k}(\hat{\mathbf{x}}_{t_k}; \mathbf{c})\mathbf{E}$ in (self-conditioned) Euler integration (8) with $D^g_{t_k}(\hat{\mathbf{x}}_{t_k}; \mathbf{c})\mathbf{E}$. Guidance scale $g = 1$ recovers the usual prediction, whereas $g > 1$ changes the velocity and hence the sampled distribution.

B.3 Self-speculative decoding

Self-speculative decoding drafts a block using the denoiser D and commits its longest accepted prefix using AR verification with f. We find it possible to build a fused operation that uses a single forward pass to simultaneously perform verification of a previously drafted block and drafting of the next block.

In each step of the fused decoding, we are given a prefix $\mathbf{c} \in V^N$, the AR prediction on the prefix $\mathbf{q} \coloneqq f(\mathbf{c}) \in (\Delta^{|V|-1})^N$, and a previously drafted block $\mathbf{p} \in (\Delta^{|V|-1})^B$ with $\hat{\mathbf{y}} = \mathsf{argmax}(\mathbf{p}^i)_{i=1}^B \in V^B$. We would like to simultaneously accept a prefix $\hat{\mathbf{y}}^{1:a}$ using AR verification with f, and also draft the next block via $D_0(\mathbf{z}; \mathbf{c} \oplus \hat{\mathbf{y}}^{1:a})$ where $\mathbf{z}$ denotes noise and $\oplus$ denotes concatenation.

Since we have the AR prediction $\mathbf{q}$ on the prefix, the verification for the first token $\hat{\mathbf{y}}^1$ can be done for free by comparing against $\mathbf{q}^N$. If $\hat{\mathbf{y}}^1 \neq \mathsf{argmax}(\mathbf{q}^N)$, then we simply replace $\hat{\mathbf{y}}^1$ by $\mathsf{argmax}(\mathbf{q}^N)$ and accept $a = 1$. Drafting then evaluates $D_0(\mathbf{z}; \mathbf{c} \oplus \hat{\mathbf{y}}^1)$, which simultaneously produces the AR prediction $f(\mathbf{c} \oplus \hat{\mathbf{y}}^1)$ on the extended prefix.

Modified document copy; original authors and local edits are listed on the cover.

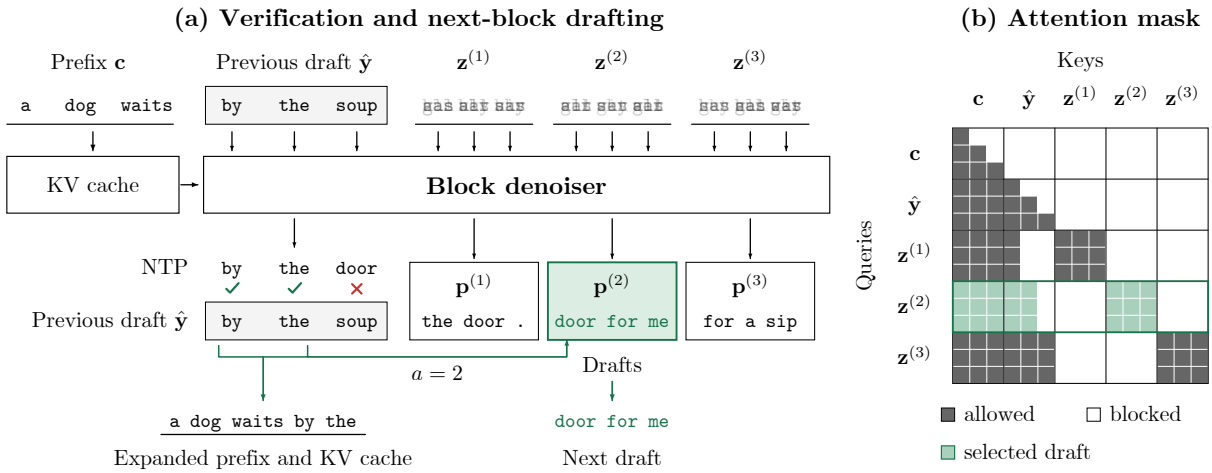


Figure 9: Fused self-speculative decoding with block size three. A single forward pass verifies previous draft with AR prediction and performs next-block drafting. This is done by decoding next drafts corresponding to every possible acceptance of the previous draft in parallel and selecting one based on verification.

Table 4: Main hyperparameters. For block size four, we use batches of 384 sequences for training, whereas for larger blocks, we use memory-matched microbatches and, when applicable, continue training from smaller-block checkpoint.

Setting	Lafa 0.5B	Lafa 7B	Lafa 8B
Model and data			
Initialization	Qwen2.5 0.5B Instruct	Qwen2.5 7B Instruct	Qwen3 8B Base
Training data / epochs / tokens	Dolci-Instruct-SFT / 5 epochs / approximately 9B		
Block sizes B	4, 8, 16	4, 8, 16	4
Maximum sequence length	512–1024	512–1024	512
adaLN	Per-layer	Shared across layers	Shared across layers
Optimization			
Optimizer	AdamW		
Learning rate	1×10^{-4}	1×10^{-5}	1×10^{-5}
Schedule / warm-up	Constant / first 1% of steps		
Adam (β_1, β_2)	(0.9, 0.999)		
Weight decay / gradient clipping / batch size	0 / 1.0 / 384 sequences		
Precision	`bfloat16`		
Distributed training	DDP	ZeRO-3	ZeRO-3
Training objective and architecture			
Training noise scale σ	4.0		
Self-conditioning probability p_{sc}	0.5		
adaLN learning rate multiplier	5.0		
Parameter EMA decay	0.999 (enabled from step 500)		
Learned tokenwise time			
Histogram bins M	200		
Histogram EMA decay β	0.99		
Update interval S	100 steps		
Flow decoding			
Inference noise scale σ	0.1		
prefix guidance scale g	8.0		

When $\hat{\mathbf{y}}^1 = \mathsf{argmax}(\mathbf{q}^N)$, we need block verification. To perform this verification and next-block drafting in a single forward pass, we construct the input to the transformer underlying D and f as a concatenation of a clean prefix $\mathbf{c} \oplus \hat{\mathbf{y}}$ and B blocks of B noise tokens, $\mathbf{z}^{(1)}, ..., \mathbf{z}^{(B)} \in \mathbb{R}^{B\times d}$, each drawn from $\mathsf{N}(0, \sigma^2 I)$. We then construct the attention mask such that the prefix $\mathbf{c} \oplus \hat{\mathbf{y}}$ attends causally to itself, while each block $\mathbf{z}^{(j)}$

Modified document copy; original authors and local edits are listed on the cover.

Table 5: Flow and flow map decoding accuracies.

			0.5B scale				*7B scale*			
Method	B	Euler steps	GSM8K	HumanEval	IFEval	Average	GSM8K	HumanEval	IFEval	Average
AR SFT	–	–	**32.0**	**35.4**	48.4	**38.6**	69.1	**78.7**	73.8	73.9
Flow decoding										
Lafa (Ours)	16	1	0.8	3.0	20.3	8.1	2.4	10.4	29.8	14.2
		2	1.7	6.1	23.5	10.4	18.7	21.3	36.2	25.4
		4	11.7	12.2	27.0	17.0	47.5	40.2	44.7	44.1
		8	18.3	14.0	30.9	21.1	60.2	54.9	56.9	57.3
		16	24.8	17.1	37.2	26.3	65.6	65.2	64.9	65.2
		32	25.9	21.3	39.9	29.1	66.9	68.3	64.1	66.5
		64	28.1	21.3	42.3	30.6	67.5	66.5	66.5	66.8
		128	28.1	20.7	44.2	31.0	68.7	67.7	65.2	67.2
	8	1	2.4	6.1	23.3	10.6	20.1	22.0	34.8	25.6
		2	9.2	12.2	23.5	15.0	47.4	41.5	42.1	43.7
		4	17.4	18.9	29.0	21.8	64.0	57.9	55.8	59.2
		8	24.8	25.6	38.4	29.6	64.8	64.0	64.5	64.5
		16	28.4	28.0	46.4	34.3	71.6	70.7	64.5	68.9
		32	29.1	28.0	45.5	34.2	72.3	72.0	71.0	71.7
		64	28.4	28.0	47.1	34.5	71.7	72.6	68.8	71.0
		128	28.7	28.7	49.0	35.5	71.0	72.6	69.3	70.9
	4	1	16.4	13.4	32.5	20.8	53.9	32.3	44.7	43.7
		2	14.7	9.1	35.1	19.7	60.0	34.8	43.8	46.2
		4	25.0	21.3	43.6	30.0	69.5	63.4	60.1	64.3
		8	27.0	22.0	46.6	31.8	73.0	71.3	69.3	71.2
		16	28.8	27.4	50.1	35.4	71.9	75.0	69.5	72.1
		32	30.4	31.1	52.9	38.1	73.8	**78.7**	73.0	75.2
		64	29.6	32.9	**53.0**	38.5	74.6	**78.7**	**74.3**	**75.9**
		128	30.1	31.7	49.4	37.1	**75.6**	78.0	73.6	75.7
Flow map decoding										
Lafa (Ours)	4	1	19.2	17.1	33.6	23.3	54.2	50.0	42.7	49.0
		2	21.1	18.3	36.2	25.2	66.4	68.3	55.1	63.3
		4	24.9	21.3	39.4	28.5	69.7	73.8	64.7	69.4
		8	28.1	26.8	40.3	31.8	70.9	73.8	68.2	71.0
		16	28.9	25.6	43.3	32.6	–	–	–	–

attends bidirectionally to itself and causally to a prefix of the prefix, $\mathbf{c} \oplus \hat{\mathbf{y}}^{1:j}$ (Figure 9). Accordingly, we build tokenwise time such that $\mathbf{c} \oplus \hat{\mathbf{y}}$ is conditioned on one and all $\mathbf{z}^{(j)}$ on zero. Since this produces the AR prediction $f(\mathbf{c} \oplus \hat{\mathbf{y}})$ on the prefix, we can determine acceptance $\hat{\mathbf{y}}^{1:a}$ as follows (Section 2.3):

$$a = \max\{j \in [B] : \hat{\mathbf{y}}^{i+1} = \mathsf{argmax}(f(\mathbf{c} \oplus \hat{\mathbf{y}})^{N+i}) \text{ for every } 1 \leq i < j\}. \tag{28}$$

Then, instead of an additional forward pass, we simply take output tokens corresponding to $\mathbf{z}^{(a)}$ since their production is computationally identical to $D_0(\mathbf{z}^{(a)}; \mathbf{c} \oplus \hat{\mathbf{y}}^{1:a})$. The next step of fused decoding then operates on extended prefix $\mathbf{c} \oplus \hat{\mathbf{y}}^{1:a}$, drafted block $D_0(\mathbf{z}^{(a)}; \mathbf{c} \oplus \hat{\mathbf{y}}^{1:a})$, and prefix AR prediction $f(\mathbf{c} \oplus \hat{\mathbf{y}}^{1:a}) = f(\mathbf{c} \oplus \hat{\mathbf{y}})^{1:N+a}$. In practice, we further reuse the key-value cache of the prefix to improve efficiency (Algorithm 4).

C Evaluation details

We describe the evaluation setup and metrics, then report detailed results including per-benchmark scores.

C.1 TPF evaluation and per-benchmark scores

Tokens per forward (TPF). Across evaluation examples j, we compute TPF as

$$\text{TPF} = \frac{\sum_j T_j}{\sum_j F_j}, \tag{29}$$

Modified document copy; original authors and local edits are listed on the cover.

Table 6: Single-token decoding. For diffusion models, we decode exactly one highest-probability token per forward. Models marked † are trained by us from the matched AR model on Dolci-Instruct-SFT for five epochs.

Method	IFEval	BBH	MMLU-Pro	GPQA	GSM8K	HumanEval	Average
0.5B scale							
Qwen2.5 SFT†	48.4	26.3	11.9	17.9	32.0	35.4	28.7
MTP† [6]	50.6	17.3	9.2	18.8	27.1	32.3	25.9
DeepSeek-MTP† [8]	53.2	24.1	**14.0**	18.5	35.5	36.0	30.2
MARS† [33]	51.3	26.6	12.4	19.4	32.8	**40.2**	30.4
SBD† [46]	**55.3**	20.6	9.2	**22.5**	27.2	30.8	27.6
BD3LM† [23]	47.1	7.5	2.0	17.9	30.6	31.7	22.8
Lafa 0.5B (Ours)†	55.1	**27.6**	12.1	21.9	**36.1**	36.6	**31.6**
7B scale							
Qwen2.5 SFT†	73.8	59.2	41.6	27.9	69.1	78.7	58.4
MTP† [6]	73.2	51.7	37.3	26.8	60.5	73.8	53.9
DeepSeek-MTP† [8]	78.0	60.7	40.6	30.1	70.3	80.5	60.0
MARS† [33]	77.8	54.6	**44.4**	26.6	73.2	**81.7**	59.7
Fast-dLLM v2†	70.6	53.5	43.1	28.6	67.2	32.9	49.3
Fast-dLLM v2 [48]	64.5	58.1	39.7	21.4	80.3	62.2	54.4
Dream [20]	57.1	**66.0**	43.6	26.6	**81.7**	58.5	55.6
Lafa (Ours)†	**78.9**	63.0	41.8	**31.0**	73.8	76.8	**60.9**
8B scale							
Qwen3 SFT†	38.1	**71.6**	56.2	29.5	62.9	**82.3**	56.8
LLaDA [45]	58.0	30.9	28.7	25.4	76.0	42.1	43.5
Efficient-DLM [21]	58.4	70.8	**61.5**	**33.7**	**87.4**	68.9	**63.5**
Lafa (Ours)†	**62.3**	70.7	56.2	31.7	77.8	79.9	63.1

Table 7: Self-speculative and parallel decoding accuracy (%) and TPF across model scales.

		IFEval		BBH		MMLU-Pro		GPQA		GSM8K		HumanEval		Average	
Method	B	Acc.	TPF	Acc.	TPF	Acc.	TPF	Acc.	TPF	Acc.	TPF	Acc.	TPF	Acc.	TPF
0.5B scale															
AR SFT	–	48.4	1.0	26.3	1.0	11.9	1.0	17.9	1.0	32.0	1.0	35.4	1.0	28.7	1.0
MARS	4	51.0	1.8	**31.7**	2.3	**13.9**	1.7	18.3	2.0	31.7	2.1	32.3	1.9	29.8	2.0
MTP	4	51.2	2.5	17.6	3.3	9.4	3.1	17.9	2.8	26.8	2.9	33.5	2.9	26.1	2.9
DeepSeek-MTP	2	51.9	1.6	23.7	1.8	12.1	1.9	18.8	1.9	35.5	1.8	35.4	1.7	29.6	1.8
BD3LM	4	46.8	1.5	8.3	1.6	1.3	1.5	19.4	1.5	29.8	1.5	29.9	1.5	22.6	1.5
SBD	4	**56.4**	2.2	20.6	3.2	9.4	2.7	23.2	2.7	26.7	2.7	31.1	2.8	27.9	2.7
PTP	4	42.3	1.7	26.0	2.5	11.9	2.0	15.0	1.9	35.3	2.3	**42.7**	2.1	28.9	2.1
Lafa (Ours)	4	55.1	2.2	27.6	3.3	12.1	3.1	21.9	2.9	36.1	2.7	36.6	2.9	31.6	2.9
	8	55.3	2.7	27.9	5.1	10.0	5.5	**23.7**	4.2	**37.8**	3.6	39.0	3.0	**32.3**	4.0
	16	54.2	3.1	27.0	6.3	9.1	9.2	18.1	4.8	36.7	3.6	39.6	3.3	30.8	5.0
7B scale															
AR SFT	–	73.8	1.0	59.2	1.0	41.6	1.0	27.9	1.0	69.1	1.0	78.7	1.0	58.4	1.0
MTP	4	73.2	1.9	51.9	3.3	38.3	2.5	27.7	2.6	60.5	3.0	73.8	3.1	54.2	2.7
DeepSeek-MTP	2	78.0	1.5	60.8	1.8	**42.5**	1.6	29.5	1.7	70.3	1.7	80.5	1.9	60.3	1.7
Fast-dLLM v2	32/8	62.8	1.9	58.1	2.3	39.2	2.2	22.3	2.2	**80.4**	2.5	63.4	1.8	54.4	2.1
Dream	Canvas	18.9	4.2	42.5	4.2	42.2	4.4	9.8	4.4	38.4	3.5	23.8	3.8	29.3	4.1
MARS	4	77.8	1.9	52.9	4.1	40.0	2.8	29.2	2.8	74.0	2.8	**81.7**	2.9	59.3	2.9
	8	76.3	2.0	59.6	5.7	41.5	3.3	30.8	3.2	71.3	3.5	81.1	3.5	60.1	3.5
	16	75.8	2.0	55.8	6.3	39.2	3.6	26.8	3.2	75.4	3.7	77.4	3.7	58.4	3.7
Lafa (Ours)	4	**78.9**	1.7	**63.0**	3.4	41.8	2.4	**31.0**	2.3	73.8	2.8	76.8	2.8	**60.9**	2.6
	8	77.5	2.8	59.1	6.3	39.4	3.7	28.1	4.0	75.3	4.8	76.2	4.6	59.3	4.3
	16	74.9	2.8	57.3	7.9	39.0	3.8	28.1	4.0	75.0	5.8	73.8	5.4	58.0	4.9
8B scale															
AR SFT	–	38.1	1.0	**71.6**	1.0	56.2	1.0	29.5	1.0	62.9	1.0	**82.3**	1.0	56.8	1.0
LLaDA	Canvas	36.8	3.2	31.0	3.2	28.8	2.7	9.4	2.7	47.2	2.7	23.8	2.5	29.5	2.8
Efficient-DLM	4	57.1	2.6	66.9	3.2	60.7	3.0	33.0	2.8	85.4	2.9	66.5	3.0	61.6	2.9
	32	58.8	2.4	70.7	3.3	**61.6**	3.1	**34.4**	2.9	**87.4**	2.9	68.9	3.0	**63.6**	2.9
Lafa (Ours)	4	**62.3**	2.4	70.7	3.4	56.2	3.2	31.7	3.1	77.8	2.6	79.9	2.9	63.1	2.9

Modified document copy; original authors and local edits are listed on the cover.

Table 8: Per-task accuracy (%) and throughput in tokens per second (TPS) at 7–8B scales, measured using one H100 GPU with batch size one. Efficient-DLM is 8B and other methods are 7B.

			IFEval		GSM8K		HumanEval		Average	
Method	Setting	Threshold	Accuracy	TPS	Accuracy	TPS	Accuracy	TPS	Accuracy	TPS
AR SFT	–	–	73.8	40.9	69.1	40.6	78.7	37.0	73.9	39.5
DeepSeek-MTP	$B = 2$	–	<u>78.0</u>	45.1	70.3	57.2	**80.5**	58.4	<u>76.3</u>	53.6
MARS	$B = 4$	0.95	77.6	42.0	70.7	59.8	<u>79.9</u>	43.3	76.1	48.4
	$B = 8$	0.95	77.3	40.9	69.1	62.7	78.0	43.1	74.8	48.9
	$B = 16$	0.95	75.0	41.1	72.9	61.3	75.0	44.6	74.3	49.0
Fast-dLLM v2	$B = 4$	0.90	65.8	52.1	81.6	66.2	67.1	56.5	71.5	58.3
	$B = 8$	0.90	65.4	60.9	80.1	98.5	57.3	71.1	67.6	76.8
	$B = 16$	0.90	61.9	65.1	77.3	116.0	56.1	78.9	65.1	86.7
Efficient-DLM	$B = 4$	0.70	58.2	31.8	<u>87.8</u>	37.1	70.1	38.4	72.0	35.8
	$B = 8$	0.70	58.2	40.7	87.7	55.6	68.9	58.5	71.6	51.6
	$B = 16$	0.70	58.4	47.2	**88.4**	70.5	67.1	75.4	71.3	64.4
	$B = 32$	0.70	59.1	49.3	87.5	74.4	63.4	84.8	70.0	69.5
	$B = 32$	0.60	58.6	56.1	82.2	87.1	55.5	96.3	65.4	79.9
	$B = 32$	0.50	52.9	71.9	72.0	102.9	42.1	111.2	55.7	95.3
Dream	$k = 1$	–	57.1	22.7	81.7	36.0	58.5	26.7	65.8	28.5
	$k = 2$	–	44.7	45.6	75.0	72.0	35.4	53.3	51.7	57.0
	$k = 4$	–	33.8	91.2	59.3	143.5	20.1	106.3	37.7	113.7
Lafa (Ours)	$B = 4$	–	**78.9**	60.9	73.8	91.7	76.8	87.2	**76.5**	79.9
	$B = 8$	–	77.5	61.1	75.3	116.1	76.2	104.3	<u>76.3</u>	93.8
	$B = 16$	–	74.9	59.7	75.0	125.2	73.8	115.7	74.5	100.2

where T_j is the number of committed tokens and F_j is the number of forward passes. AR has TPF one by definition. For Lafa and Efficient-DLM, F_j counts the number of fused verification-and-redraft forward passes in addition to one forward pass for initial drafting, excluding prompt prefill and cache refreshes for Efficient-DLM. For Fast-dLLM v2, F_j includes top-level denoising, prefill, and cache update forward passes.

Per-benchmark scores. Table 5 gives the per-benchmark scores of flow and flow map decoding used to construct Figure 6. Table 6 expands single-token decoding Table 1 into per-benchmark accuracies. Table 7 gives the per-benchmark values underlying the parallel decoding results in Figure 7 including our self-speculative decoding results. All methods in Figure 7 decode by drafting and verifying, and the verifier follows the native configuration of each method. Causal block predictors verify against greedy AR, MARS uses its released batched verifier, and Efficient-DLM uses its exact verifier. Fast-dLLM v2, Dream, and LLaDA instead verify against their deterministic one-position decoding trajectories.

C.2 Decoding throughput

Figure 8 shows end-to-end generation throughput measured on a single H100 GPU with batch size one with `bfloat16` inference and a 32-token warm-up. We report tokens per second (TPS), which is the total generated tokens divided by total generation time. Table 8 reports the per-task values underlying Figure 8, and method-specific settings are given below.

- **AR SFT.** Qwen2.5 7B AR SFT uses standard HuggingFace greedy generation and emits one new token per decoding forward.
- **Lafa (Ours).** For each block size $B \in \{4, 8, 16\}$, we use the corresponding trained checkpoint with self-speculative decoding. A one-step denoiser draft of the next block is verified against the model's own greedy AR predictions. The fused pass packs the clean draft with B noisy successor blocks, one for each possible accepted-prefix length. Clean draft positions attend causally to produce the verification logits, while each successor block attends to the prefix and its candidate accepted draft prefix and bidirectionally within

Modified document copy; original authors and local edits are listed on the cover.

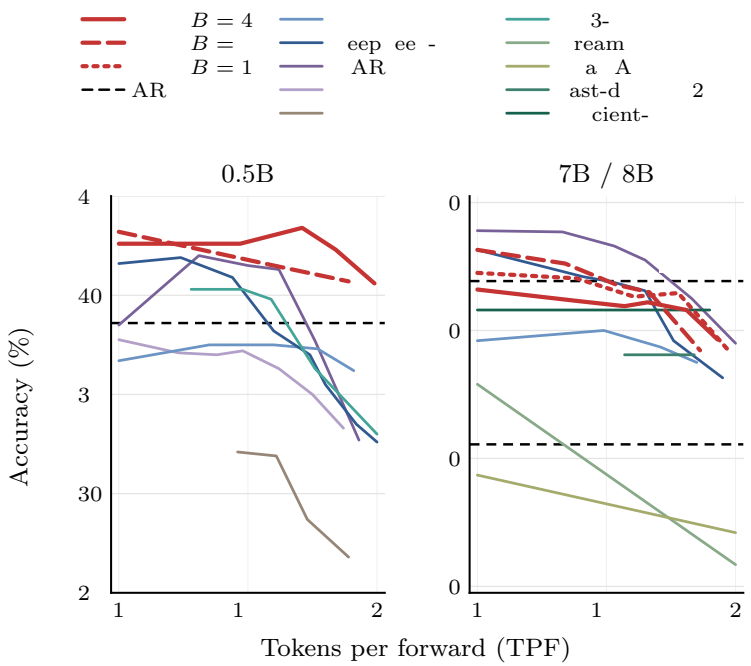


Figure 10: Confidence decoding, averaged over GSM8K, HumanEval, and IFEval. In the right panel, Lafa uses 8B for $B = 4$ and 7B for $B \in \{8, 16\}$. Efficient-DLM ($B = 32$) and LLaDA are 8B, and other baselines are 7B.

the block. After selecting the longest verified prefix, we retain its KV states and use the corresponding successor block as the next draft, avoiding a separate redrafting forward.

- **DeepSeek-MTP.** We train 7B scale DeepSeek-MTP with one DeepSeek-style auxiliary multi-token-prediction stage ($D = 1$ [8]). Greedy verified self-speculation compares the multi-token proposal against the base model's greedy AR prediction.
- **MARS.** We use the released MARS Qwen2.5 7B checkpoints [33] for $B \in \{4, 8, 16\}$. We report results using the paper's probability-threshold decoder at threshold 0.95, with its sliding KV cache. Each round commits the leftmost proposal, then commits the longest consecutive continuation whose top-token probabilities remain at least the confidence threshold.
- **Fast-dLLM v2.** We use the released Fast-dLLM v2 7B checkpoint [48] with its native confidence decoder at threshold 0.9. Block caching is disabled, and the global and sub-block sizes are both set to $B \in \{4, 8, 16\}$.
- **Efficient-DLM.** We evaluate Efficient-DLM 8B [21] with its native confidence-threshold decoder at threshold 0.7 for $B \in \{4, 8, 16, 32\}$, together with the $B = 32$ speed–quality sweep at confidence thresholds in $\{0.5, 0.6\}$. We use the model's native prompt format over Qwen chat templates.
- **Dream.** We use Dream-v0 7B Instruct [20] with its native entropy-based unmasking, targeting a mean of $k \in \{1, 2, 4\}$ filled canvas positions per denoising forward. For throughput, we follow Efficient-DLM [21] and compute TPS as the total number of generated canvas tokens divided by total generation time.

C.3 Confidence decoding

As an alternative to self-speculative decoding, we follow MARS [33] and accept the longest left-to-right prefix of a one-step flow draft whose token confidence exceeds a threshold λ (Appendix B.3), and apply the same rule to block-autoregressive baselines. We implement it with the fused decoding of Algorithm 4, where $\hat{\mathbf{y}}^{1:a}$ is instead the longest prefix of $\hat{\mathbf{y}}$ with confidences $(\mathbf{p}^i)_{\hat{\mathbf{y}}^i} \geq \lambda$ (where we always set $a \geq 1$). Since a is

Modified document copy; original authors and local edits are listed on the cover.

known before the forward pass, only the block $\mathbf{z}^{(a)}$ is needed instead of all B blocks. To apply this strategy, Dream and LLaDA use their native global entropy or confidence ordering over masked positions, whereas Fast-dLLM v2 applies the threshold within the active sub-block. In Figure 10, we show an accuracy–TPF tradeoff measured by sweeping over confidence thresholds. At 0.5B, Lafa with block size four attains the highest peak accuracy among all methods, including AR SFT, and its accuracy remains at or above that of pure AR at high thresholds. At 7B, Lafa exceeds AR SFT at high thresholds, while MARS, which is trained for confidence decoding, degrades more gracefully as the threshold is lowered. At 8B, Lafa remains above AR SFT across the tested range and on par with Efficient-DLM.

Since AR verification avoids the degradation at lower thresholds without tuning a threshold, we use self-speculative decoding as our default in the main text.

Modified document copy; original authors and local edits are listed on the cover.